INTERNATIONAL HELLENIC UNIVERSITY

Substituting MLPs in Transformers with Kolmogorov-Arnold Networks: A New Direction in Deep Learning.

Orfeas Ilias Gkonis

SID: 3308240008

SCHOOL OF SCIENCE & TECHNOLOGY

A thesis submitted for the degree of
Master of Science (MSc) in Data Science

February 2026
THESSALONIKI – GREECE

Substituting MLPs in Transformers with Kolmogorov-Arnold Networks: A New Direction in Deep Learning.

Orfeas Ilias Gkonis
SID: 3308240008

Supervisor:
Prof. Panayiotis Bozanis

Supervising Committee Members:
Dr. Leonidas Akritidis
Dr. Dimitrios Karapiperis

SCHOOL OF SCIENCE & TECHNOLOGY
February 2026
THESSALONIKI – GREECE

Contents

List of Figures

List of Tables

Thanks To

I would like to express my sincere gratitude to my supervisor, Panayiotis Bozanis, whose guidance and insightful feedback were invaluable throughout the course of this research. I am also deeply thankful to Leonidas Akritidis for his continuous support and willingness to provide guidance whenever needed.

Finally, I extend my heartfelt appreciation to my family, whose support and encouragement made the completion of this work possible.

Abstract

Transformers have established themselves as a key framework in today's deep learning systems,achieving state-of-the-art performance across various domains such as natural language processing, computer vision, and time-series forecasting.A key component of Transformer architectures is the multi-layer perceptron (MLP) which contributes significantly to the model's expressiveness and computational load. This thesis investigates the potential replacement of MLPs with the Kolmongorov-Arnold Networks(KANs), a novel class of networks that leverage the Kolmogorov-Arnold representation theorem to model complex multivariate functions through simpler univariate transformations and learned basis functions. We hypothesize that KANs can perform better than MLPs in certain datasets and models, and that their integration into Transformer architectures is worth exploring experimentally. The experiments consisted of three different datasets, two of them were image datasets and one text dataset. A total of four different Transformers were tested, Vision Transformer (ViT), Swin, EvA and BERT and two different KAN layer variants were tested in replacement of MLPs.

Keywords: Transformers, MLPs, KANs

Orfeas Ilias Gkonis
February 2026

1 Introduction

Before the introduction of Transformer architectures, deep learning in both natural language processing and computer vision was dominated by models that relied heavily on inductive biases. In NLP, Recurrent Neural Networks (RNNs) and particularly Long Short-Term Memory (LSTM)[1] and Gated Recurrent Unit (GRU)[2] architectures, formed the backbone of most sequence modeling approaches. These models were processing the input sequentially which limited parallelizable training and made it difficult to capture long range dependencies. In computer vision, Convolutional Neural Networks (CNNs) were the industry standard[3], leveraging locality and weight sharing to efficiently extract spatial features, but often requiring deep and carefully engineered architectures to model global context. While both RNN and convolutional based architectures achieved significant success, they faced scalability and efficiency constraints. These limitations, along with the growing availability of large datasets and computational resources, lead to the innovation of more flexible model architectures, being able to capture long range dependencies, ultimately leading to the development of Transformer based models.

Transformer is a deep neural network that employs a self-attention mechanism to comprehend the contextual relationships within sequential data. Unlike Convolutional Neural Networks or updated versions of Recurrent Neural Networks, Transformer models excel in handling long dependencies between input sequence elements and enable parallel processing [4]. Transformer was originally proposed as a sequence-to-sequence model[5] for machine translation.After researchers' experimentation it was shown that Transformer can achieve state-of-the-art performance on various tasks. The original Transformer architecture demonstrated that self-attention alone,without recurrence or convolution was sufficient to achieve superior performance in machine translation. This breakthrough marked the beginning of a significant paradigm shift in deep learning. Shortly after, models such as BERT, GPT, and RoBERTa expanded the Transformer architecture to pre-training large scale language models, enabling contextual understanding, bidirectional encoding, and generative capabilities.Over time, Transformers evolved beyond NLP, becoming foundational in multimodal and vision tasks. Vision Transformer (ViT)

showed that Transformers could outperform Convolutional Neural Networks when trained on large image datasets, leading to widespread adoption in computer vision applications. To this day, Transformers consist the backbone of all the modern AI applications such as ChatGPT due to their ability to learn long range dependencies from the data, their computational efficiency,explainability and flexible architecture.

Multi-layer Perceptron (MLP) or fully connected (FC) network , on the other hand, is another important part of Transformers and is composed of multiple linear layers and non linear activations stacked together [6], [7] and is responsible for the computational load and expressiveness of the model. What makes it so popular is that, theoretically, they can approximate any function, assuming that there are enough neurons available [8]. Much progress has been made in improving performance and in understanding how these neural networks operate. However, the need for additional improvements in training these networks still exists since the training process is very chaotic in nature[9].Despite their usefulness,MLPs:

- may still struggle to model complex functions.For example, when using ReLU-like activation, a two-layer MLP may struggle to fit periodic functions [10].
- consume almost all non-embedding parameters and are typically less interpretable (relative to attention layers) without post-analysis tools [11].

Kolmongorov-Arnold networks (KANs) are proposed as an alternative to MLPs. Whereas MLPs are inspired by the universal approximation theorem, KANs are inspired by the Kolmogorov-Arnold representation theorem [12],[13]. Like MLPs, KANs have fully-connected structures. However, while MLPs place fixed activation functions on nodes (“neurons”), KANs place learnable activation functions on edges (“weights”). As a result, KANs have no linear weight matrices at all: instead, each weight parameter is replaced by a learnable 1D function parametrized typically as a B-spline [14], allowing them to learn activation patterns dynamically.

The initial introduction of KANs generated significant enthusiasm within the machine learning community, with proponents highlighting their potential advantages in function approximation, interpretability, and computational efficiency[15].Early studies have shown encouraging results [16],showing hope in datasets with many instances, indicating their potential to handle complex datasets effectively [17],so researchers proceeded into further experimentation. However, as the field developed, the complete picture has emerged. New studies indicated that KANs advantages may be more limited than initially suggested [18]. Specifically,while KANs demonstrate superior performance in symbolic regression tasks, they consistently underperform traditional MLPs in standard machine learning benchmarks including computer vision, natural language processing tasks. KANs in their standalone form, offer comparable accuracy to MLPs, but come at a significantly higher

parameter count and computational cost[19]. Computational overhead analysis shows KANs can be 10 to 100 times slower than equivalent MLPs, severely limiting their practical deployment.

This study compares MLPs and KANs across both theoretical and practical dimensions. We experimented with KANs on image and text classification tasks on three datasets total.One was a text dataset while the other three were image datasets. Four Transformers were used in total, Vision Transformer(ViT), Swin, EvA and BERT. All the experiments where implemented in PyTorch and PyTorch Lightning and the models where evaluated based on 5-fold cross-validation test accuracy. Additionally, two KAN layer types where tested: KAN Linear[18] and GR-KAN[10].

The following of this thesis is consisted of: The theoretical background which provides the Kolmogorov-Arnold theorem, introduces the architecture and training methods of KANs, including their spline-based and the group rational variant representation. The Methodology section: presents the experiments the the implementation details of all the models.Finally, in the last section we present our conclusions from the experimentation process, regarding the efficiency and performance of KANs compared to MLPs in Transformers.

2 Theoretical Background

Since 2006, deep structured learning, or more commonly called deep learning or hierarchical learning, has emerged as a new area of machine learning research[20], emerging as a solution to complex problems in computer vision and natural language processing. At its core there are CNNs, that are particularly effective at learning spatial hierarchies of features from image data and RNNs that process the data sequentially. Building upon these advances, models like ViT, Swin and BERT have emerged to improve performance in image and text classification.

This chapter introduces the foundation of deep learning and Transformers, followed by an in depth analysis of the Kolmogorov-Arnold theorem and its implementation to the KAN and GR-KAN layers.

2.1 Deep Learning

Deep Learning is a sub-field of machine learning that is based on learning several levels of representations, corresponding to a hierarchy of features or factors or concepts, where higher-level concepts are defined from lower-level ones, and the same lower level concepts can help to define many higher-level concepts. Deep learning is part of a broader family of machine learning methods based on learning representations. An observation (an image) can be represented in many ways (a vector of pixels), but some representations make it easier to learn tasks of interest (is this the image of a human face?) from examples, and research in this area attempts to define what makes better representations and how to learn them[21].

The rise of deep learning in recent years has been driven by the emergence of GPUs and the availability of large datasets and they were greatly enhanced by the development of open source, flexible software platforms with automatic differentiation, enabling remarkable achievements across various machine learning tasks. Its coninuous evolution has enhances fields such as image super-resolution, object detection and image recognition. Notably, deep learning has surpassed human performance in tasks like image classification, reinforcing its impact on on data-driven decision making.

Although the intuition that deeper neural networks could be more powerful

pre-dated modern deep learning techniques[22, 23], it was a series of advances in both architecture and training procedures[24], which ushered in the remarkable advances which are associated with the rise of deep learning. It is hypothesized that deep networks excel because they exploit a particular form of compositionality in which features in one layer are combined in many different ways to create more abstract features in the next layer[25].

However,in most of the applied deep learning models, there is a lack of explainability, meaning that even though their inference from data might work well, the mechanisms behind the predictions are not well understood. As the ambition in all sciences it to understand causal relationships rather than pure correlations, this might neither be satisfying nor lead to further deeper understandings in corresponding fields. Without understanding the details of a model, potential robustness issues might not be realized either[26]. Deep neural networks continue to be criticised for a lack of rigorous mathematical foundations and theoretical insight into key phenomena such as generalization, optimization, and robustness. Several works have noted that the current understanding of deep learning is incomplete: for example, basic behaviors like overparameterization and double descent lack unified theory, and deep models are often treated as black boxes with limited interpretability, rather than principled, explainable systems[27, 28]

Deep learning research is destined for significant evolution, with the current literature identifying key future directions such as algorithmin efficiency, enhanced architectual design and learning paradigms beyond supervised learning. For example, compute-efficient training techniques and bottleneck mitigation strategies are highlighted as promising research areas[29]. While comprehensive reviews stress the need for continued innovation in model architectures, training methods, and scalability[30, 31].

2.2 Transformers

Transformer[32] was firstly proposed in 2017 as a sequence to sequence model and consists of an encoder and a decoder, each of which is a stack of L identical blocks. Each encoder block is mainly composed of a multi-head self-attention module and a position-wise feed-forward network. For building a deeper model, a residual connection[33] is employed around each module, followed by Layer Normalization. Compared to the encoder blocks, decoder blocks additionally insert cross-attention modules between the multi-head self-attention modules and the position-wise FFNs. Furthermore, the self-attention modules in the decoder are adapted to prevent each position from attending to subsequent positions [34].

Transformers adopt the attention mechanism with Query-Key-Value (QKV)

model. Given the matrix representations of queries **Q**$\in \mathbb{R}^{NxD_k}$, keys **K**$\in \mathbb{R}^{MxD_k}$ and values **V**$\in \mathbb{R}^{MxD_v}$, the scaled dot product attention used by Transformer is given by:

$$Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{D_k}})V = AV \tag{2.1}$$

where M,N denote the lengths of queries and keys (or values),D_k,D_v denote the dimensions of keys (or queries) and values, A is often called attention matrix. The dot-products of queries and keys are divided by $\sqrt{D_k}$ to alleviate gradient vanishing problem of the softmax function.

Instead of applying a single attention function, Transformer applies multi-head attention. The queries Q, values V and keys K are projected into different dimensions with H different projections learnt. For each projected query, key and value an output is computed with the attention function.

$$MultiHeadAttn(Q,K,V) = Concat(head_1,...,head_H)W^O \tag{2.2}$$

, where $head_i = Attention(QW_i^Q, QW_i^K, QW_i^V)$ and W are the learned linear projection matrices.

In Transformers there are three types of attention mechanisms:

1. **Self attention**: In the Transformer encoder, where we set $Q = K = V = X$, X is the output of the previous layer.

2. **Masked Self attention**: In the Transformer decoder, this is done usually by applying a mask function to the unnormalized attention matrix $\hat{A} = exp(\frac{QK^T}{\sqrt{D_k}})$. This kind of self-attention is often referred to as autoregressive or causal attention.

3. **Cross attention**: The queries are projected from the outputs of the previous (decoder) layer, whereas the keys and values are projected using the outputs of the encoder.

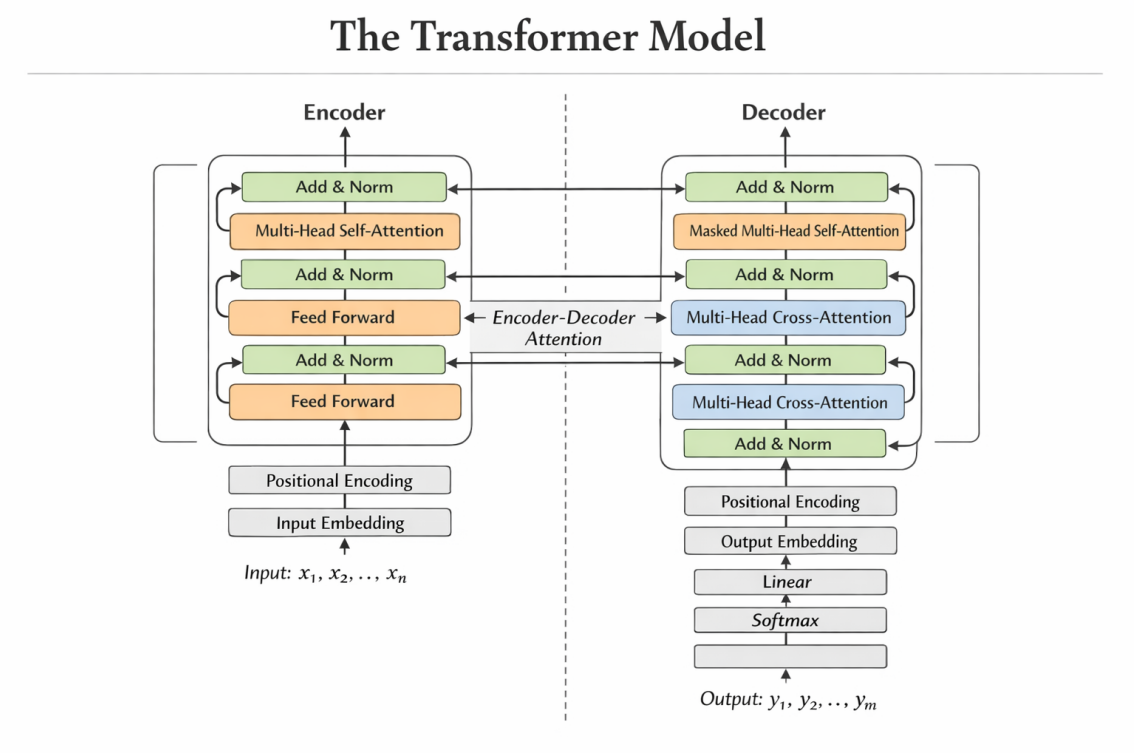

Figure 2.1: The architecture of Transformer

Despite their best-in-slot performance, computing the whole nxn attention matrix results in an $O(n^2)$ time complexity increasing latency and compute costs as the input length n grows. Additionally, Transformers need a lot of data in order to achieve the promised performance, leading into potential memory issues[35].

2.2.1 The Multi-layer Perceptron

The Multi-layer Perceptron is the most utilized model in neural networks being associated with the backpropagation algorithm in order to update the weights of the model. The definition of architecture in MLP networks is a very relevant point, as a lack of connections can make the network incapable of solving the problem of insufficient adjustable parameters, while an excess of connections may cause an over-fitting of the training data[36].

Multi-layer Perceptron is an Artificial neural network(ANN) which consists of multiple layers and neurons feedforward. The architecture of Multi-layer Perceptron is composed of: An input layer, that receives the input features and passes them to the next layer. The hidden layers, that perform the calculations using weighted connections and non-linear activation functions. The hidden layer can be more than one. The output layer that produces the final prediction.

We can visualize it as a finite acyclic graph [37]. Each layer is composed of nodes and when every node is connected to every node in the subsequent layers

then the network is called fully connected. When dealing with regression and single classification tasks a single output neuron is used and when we are dealing with multi-class classification tasks the number of output neurons is determined by the number of classes. The connections are defined by learnable weights that

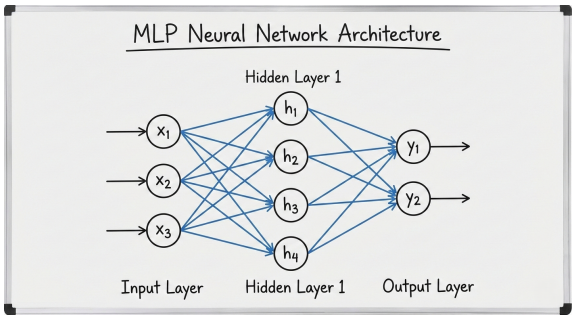

Figure 2.2: The architecture of MLP consisted of three layers total, three inputs and two outputs

control how strongly each input feature contributes to the following layer. Bias terms provide an additional offset to the activation functions, enhancing the model's ability to represent complex patterns. To be exact, each neuron computes the weighted sum of the inputs and then applies the activation function to produce the output.

With mathematical terms: For input data $x_1, x_2, ..., x_n$, weights $w_1, w_2, ..., w_n$ and bias b, each neuron calculates $z = \sum_{i=1}^{n} w_i * x_i + b$. The the activation function σ is applied to z produce the output.

By applying this function driven mapping, MLPs can capture intricate patterns in the data, which underlies their capacity to serve as universal approximators according to the Universal Approximation Theorem[38]. This theorem states that MLPs can approximate any continuous function if the activation function is non-polynomial and the number of neurons and hidden layers is sufficient enough.

The learning of MLP is achieved through the minimization of an error function, between the network's outputs and the desired outputs[39]. At the start of training the MLP's weight valus get initialized randomly and perform a forward pass to compute the neuron activations from input to output, calculating the value of the error function. Then the network performs backpropagation computing the gradients of the error function. At last, the weights and biases are updated with respect to gradient descent[40].

Some widely used error functions are:

- **Cross Entropy Loss**: $C = -\sum_{i=1}^{n} y_i log(\hat{y}_i)$. Cross-entropy loss is a way to measure how close a model's predictions are to the correct answers in classification problems.It helps train models to make more confident and

accurate predictions by rewarding correct answers and penalizing wrong ones. This makes it a key part of building reliable machine learning classifiers.

- **MSE Loss**:$M = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$. Mean Squared Error (MSE) is one of the most common metrics used for evaluating the performance of regression models. It measures the average of the squares of the errors—that is, the average squared difference between the predicted and actual values. It provides a way to quantify how much error exists in a model's predictions.

Some famous activation functions are:

- **ReLU** (Rectified Linear Unit)[41]: $\sigma(x) = max(0,x)$, is a popular activation functions used in neural networks, especially in deep learning models. It has become the default choice in many architectures due to its simplicity and efficiency.Although it seems like a piecewise linear function, ReLU is still a non-linear function. ReLU offers computational advantages in terms of backpropagation, as its derivative is simple—either 0 (when the input is negative) or 1 (when the input is positive). This helps to avoid the vanishing gradient problem[42], which is a common issue with sigmoid or tanh activation functions.

- **Sigmoid**:$\sigma(x) = \frac{1}{1+e^{-x}}$. Sigmoid is used in modern machine learning as an activation function to model binary or multi labels classification tasks,smoothing outputs, and introducing non-linearity into models.

- **Softmax**:$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$. The softmax activation function converts logits into a probability distribution where the sum equals to 1. and amplifies larger values making the model's confidence more pronounced. Therefore, softmax is usually used in the classification head of neural network, because it transforms raw logits into an interpretable probability.

- **SiLU** (Sigmoid Linear Unit)[43]:$\sigma(x) = xs(x)$. Where $s(x)$ is the sigmoid function. Was introduced as an alternative to ReLU. Unlike ReLU, SiLU has a smooth curve and can is bounded to -1. Additionally, SiLU has a negative derivative which acts like a regularizer and prevents exploding gradients. The benefits of SiLU in comparison to ReLU are that, ReLU neurons can become inactive (stuck at zero) if the weights and biases are not set properly. SiLU's smoothness helps avoid this problem and SiLU works well with batch normalization.

- **GeLU** (Gaussian Error Unit)[44]: $\sigma(x) = 0.5x(1 + tanh(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3))$. The key advantage of GeLU lies in its smoothness and differentiability across

the entire real line. This smoothness facilitates training, as gradient-based optimization algorithms can easily navigate the function's landscape. It also mitigates the vanishing gradient problem and speeds up convergence during training. However, GeLU comes with increased computational complexity and its performance might be task specific.

2.3 The Kolmogorov-Arnold Theorem

The theoretical basis of Kolmogorov–Arnold Networks (KANs) is rooted in the Kolmogorov–Arnold representation theorem. Originally introduced by Andrey Kolmogorov and later refined by Vladimir Arnold, this theorem asserts that any multivariate continuous function can be expressed as a superposition of continuous univariate functions. This result forms the conceptual backbone of KANs' unique architectural design. In essence, the theorem implies that a complex function involving several variables can be decomposed into a set of simpler one-dimensional functions, which are then combined through addition to reconstruct the original multivariate function[45].

.

$$f(x_1,...,x_n) = \sum_{q=1}^{2_{n+1}} \phi_q(\sum_{p=1}^{n} \phi_{p,q}(x_p)) \tag{2.3}$$

In this context, the multivariate function is $f(x_1,...,x_n)$ and it can be decomposed into the functions

$$\phi_1 + ... + \phi_n = \sum_{i=1}^{n} \phi_q(x_q) \tag{2.4}$$

and then this output goes through f again. Unlike MLPs, which use fixed activation functions at each node, KANs employ learnable activation functions along the edges. As a result, KANs do not rely on traditional linear weights. Instead, every weight parameter is replaced by a univariate function, typically implemented as a spline,allowing them to focus on these functions instead of the matrix based operations in MLPs.

2.4 Kolmogorov-Arnold Networks

Kolmogorov-Arnold Networks (KANs) take advantage of the Kolmogorov-Arnold representation theorem,which establishes that any continuous multivariate function can be represented by a sum of univariate functions. KAN decomposes high-dimensional inputs into univariate transformations, allowing effective approximation of complex multivariate functions.Given an input vector $\mathbf{x} = (x_1,...,x_n)$ with

dimension $d_{in} = n$, KAN models the function $f : R^{d_{in}} \to R^{d_{out}}$ as follows:

$$f(x) = \Phi \circ x = [\sum_{i=1}^{d_{in}} \phi_{i,1}(x_i), ..., \sum_{i=1}^{d_{in}} \phi_{i,d_{out}}(x_i)] \tag{2.5}$$

. Where $\phi_{i,j}$ represent the univariate transformations of the input and Φ represents the composite mapping applied across the input dimensions. KANs typically use B-spline functions as activation functions because of their smoothness and flexibility, which offer improved capability for modeling nonlinear relationships. In order to approximate any function with KANs, the following steps are executed:

1. **Univariate Transformations**: Every input x_i is passed through a univariate function $\phi_{p,q}(x_i)$ producing values $z_q = \sum_{q=1}^{d} \phi_{p,q}(x_q)$.

2. **Non Linear Activation with B-Splines**: Each result z_q is passed through a linear combination of an activation function and a B-spline function:

   $$z'_q = \phi_q(z_q) = w_b * silu(z_q) + w_s * spline(z_q) \tag{2.6}$$

   , where $spline(x) = \sum_i c_i B_i$,with B_i as the basis functions of the B-spline.These transformations happen on the edge of each passthrough from layer to layer, instead of each node which happens in MLP.

3. **Final output**:The network summarizes the outputs from the hidden layers into the final function.

   $$y = \sum_{p=1}^{2d+1} \phi_p(\sum_{q=1}^{d} \phi_{p,q}(x_q)) \tag{2.7}$$

   [46]

KANs offer several key advantages to the machine learning community such as:

- KANs come from an established mathematical background, specifically the Kolmogorov-Arnold theorem, which supports their ability to represent any continuous multivariate function as a combination of univariate functions.This makes KANs particularly well-suited to tasks that involve complex compositional structures. In consequence, they offer interpretability which is lacking on latest models.

- The use of smooth splines in KANs helps to enforce smoothness in the learned functions, which is advantageous in scenarios where overfitting to noisy data could obscure the true underlying relationships.

- KANs achieve outstanding performance in PDE solving [47] and symbolic regression [48] offering faster convergence, in comparison to MLPs.

However, KANs come with some critical disadvantages too. Specifically, they usually come with slower training because we the activation functions are learnable and they are placed on each edge of the network.In mathematical terms, there are total of $O(N^2L(G+K)) \sim O(N^2LG)$ learnable parameters,where N is the number of layers, L is the depth, k is the order of the spline (usually is 3) on G intervals. In contrast, MLPs have $O(N^2L)$ learnable parameters. Additionally, B-splines functions are not parallel computing friendly, because they require recursive computation, which slows down significantly the implementation process. Lastly, even though the weight initialization of KANs is similar to that of MLPs, it does not meet the requirements for faster convergence. With higher order splines, the variance instability might increase. Thus, the default initialization opposes the essential variance-preserving principle. This misalignment can lead to instability and degraded performance during the training process.

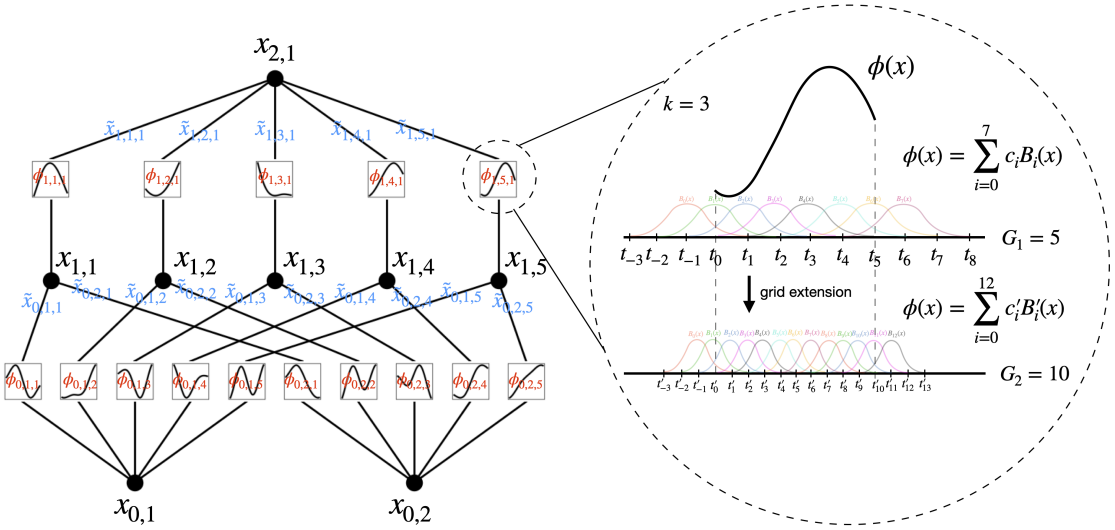

Figure 2.3: A figure of a three layer KAN with two inputs and one output

2.4.1 B-splines

Splines are used in statistics in order to mathematically reproduce flexible shapes. Knots are placed at several places within the data range, to identify the points where adjacent functional pieces join each other. Smooth functional pieces (usually low-order polynomials)are chosen tofit the data between two consecutive knots. The type of polynomial and the number and placement of knots is what then defines the type of spline[49].

B-splines, or basis splines, are an important tool in numerical analysis and computer graphics for curve fitting and data smoothing. They offer a flexible way to represent curves and surfaces through piecewise polynomial functions. A

B-spline is a type of spline function that provides minimal support with respect to a given degree, smoothness, and domain partition. In simpler terms, they are piecewise polynomial functions defined over a sequence of intervals known as knots. B-splines are very attractive as base functions for ("nonparametric") univariate regression[50]. A B-spline of order q has the following properties:

- it consists of $q+1$ polynomial pieces, each of degree q.
- the polynomial pieces join at q inner knots.
- at the joining points, derivatives up to order q-1 are continuous.
- the B-spline is positive on a domain spanned by $q+2$ knots; everywhere else it is zero.
- except at the boundaries, it overlaps with $2q$ polynomial pieces of its neighbors.
- at a given x, $q+1$ B-splines are nonzero.

2.4.2 Group-Rational KAN

Group-Rational KAN is KAN layer variant that changes the B-splines as a base function of the layer with rational functions[51]. Specifically, each $\phi(x)$ is parametrized as a fraction of polynomials $P(x), Q(x)$ of order m, n respectively.

$$\phi(x) = wF(x) = w\frac{P(x)}{Q(x)} = w\frac{a_0 + a_1 x + ... + a_m x^m}{b_0 + b_1 x + ... + b_n x^n} \tag{2.8}$$

,where a_m, b_n are the coefficients of the polynomials and w is a scaling factor. The learnable parameters are a_m, b_n, w and the degrees m, n of the polynomials can be treated as hyper-parameters.

The advantages of rational functions over B-splines are the following. First, from an efficiency standpoint, polynomial evaluation requires only simple, highly parallelizable operations, making rational functions computationally attractive for large-scale models. Second, from a theoretical perspective, rational functions can approximate a broader class of functions, including those with singularities or sharp transitions, more effectively and accurately than polynomials [52, 53]. Because B-splines are composed of local polynomial segments, rational functions provide a theoretical advantage in capturing complex behaviors. Third, from a practical perspective, rational activations have already been applied successfully as activation functions in neural networks [54]. For these reasons, adopting rational

functions as the foundational components of KAN layers to improve expressiveness, stability, and computational efficiency.

In addition to the rational functions, instead of learning a unique base function $\phi(x)$ for each input-output pair, the edges are grouped. With this method the number of parameters and computation are reduced. Assuming that the number of groups is g, then the number of parameters of Group-Rational KAN (GR-KAN) becomes $d_{in} * d_{out} + d_{out} + (m + n * g)$ and GR-KAN can be parametrized as

$$GR - KAN = \Phi \circ x = [\sum_{i=1}^{d_{in}} w_{i,1} F_{\frac{i}{dg}}(x_i) ... \sum_{i=1}^{d_{out}} w_{i,d_{out}} F_{\frac{i}{dg}}(x_i)] \tag{2.9}$$

The original KAN architecture requires $d_{in} * d_{out}$.unique activation functions. With this grouping strategy, only g unique functions are needed, reducing the parameter count to a constant overhead comparable to that of a standard MLP.Beyond parameter savings, this grouping also lowers computational cost. Each input channel evaluates its activation function ϕ only once and this result is shared across all associated output channels. In contrast, the original KAN must compute a distinct $\phi_{i,j}$ for every output channel j, leading to substantially higher computation.

GR-KAN introduces Variance-Preserving Initialization, it aims to initialize the values for a_m, b_n and w in Group-Rational KAN to ensure variance-preserving behavior across the network. At its core prevent the growth or reduction of activation magnitudes throughout the layers, thereby maintaining stability. To make the process manageable, Instead of sampling w, a, b jointly, they are sample sequentially. Initially, a, b are determined such that F fits established activation functions like ReLU, GELU or SILU. Once a, b are set,

$$a = \frac{E[F(x^2)]}{Var[x]} \tag{2.10}$$

is estimated numerically, assuming $x \sim N(0, 1)$. Then, a is used to initiallize w from the distribution $N(0, \frac{a}{d_{in}})$

2.5 Vision Transformer

Vision Transformer (ViT)[55] represents a paradigm shift in computer vision by successfully adapting the Transformer architecture, originally designed for sequential data in natural language processing, to the domain of image recognition. Its core premise is the treatment of an image not as a spatially structured grid of pixels but as a sequence of flattened, linearly embedded patches, which are then processed by a standard Transformer encoder.To summarize the Vision Transformer's

architecture, the following steps are applied:

1. **Input Image Processing**: The input image is divided into patches, which are flattened and embedded using a linear projection.
2. **Positional Encoding**: Positional encodings are added to the patch embeddings to retain spatial information.
3. **Transformer Encoder**: The patch embeddings (along with the CLS token) are passed through multiple Transformer encoder blocks, which include multi-head self-attention and feed-forward networks.
4. **Classification Head**: The CLS token's output is extracted and fed into an MLP for final classification.

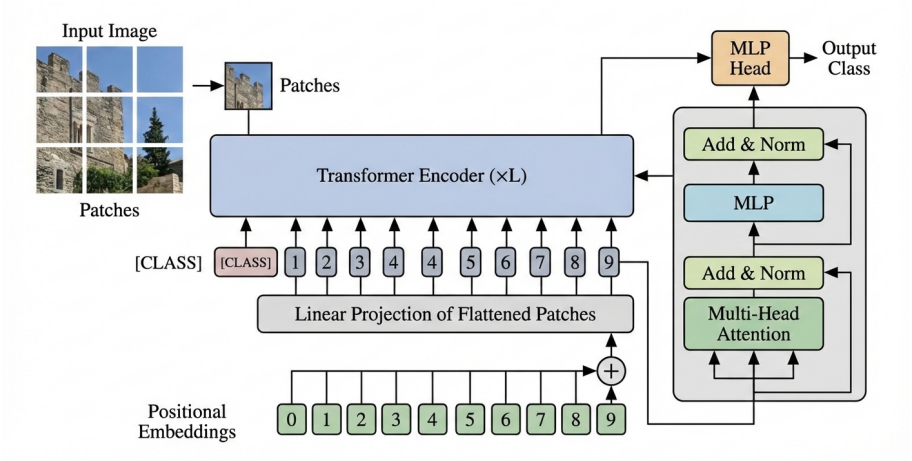

Figure 2.4: Vision Transformer's architecture

It should be emphasized that ViT uses solely the standard Transformer encoder aside from a modification in the layer normalization placement, whose output is fed into an MLP classification head. ViTs are generally pretrained on extensive datasets and later fine tuned on smaller ones for specific downstream applications[56]. With that said, because Transformers lack some inductive biases, they do not generalize when they are trained on insufficient amounts of data. However, when the models are trained on large scale datasets (14 million to 300 million images) the inductive bias can be surpassed. When pre-trained at big scale, Transformers' performance can be outstanding and surpass other models' performance with less data points.

2.6 Swin Transformer

Inspired by Vision Transformer's performance researchers experimented with alternative ViT architectures [57] for better image classification. It was found that

ViT's architecture is unsuitable for use as a general-purpose backbone network on dense vision tasks or when the input image resolution is high, due to its low-resolution feature maps and the quadratic increase in complexity with image size.Swin Transformer (Shifted Window Transfomrmer) is proposed the best offer between the speed-performance tradeoff on image classification[58].The model builds a hierarchical feature representation, enabling it to process visual information across multiple scales. This structure allows the Swin Transformer to effectively capture both fine-grained and large-scale patterns, making it well suited for dense prediction tasks. Moreover, the shifted-window mechanism restricts self-attention to non-overlapping local windows, substantially increasing computational efficiency. In contrast to traditional Transformer models that use fixed-size feature maps and incur quadratic complexity, the Swin Transformer achieves linear computational complexity with respect to image size, making it a scalable and efficient choice for high-resolution vision applications.

Swin's architecture consists of four sequential stages. The process starts with the Patch Partition layer, which divides the input RGB image into non-overlapping patches. Each patch is subsequently flattened and mapped to a feature vector. A Linear Embedding Layer comes after and it is followed by a number of consecutive Swin Transformer blocks which they capture meaningful relationships between the data. After that, a Patch Merging Layer is applied allowing the model to build feature representations.

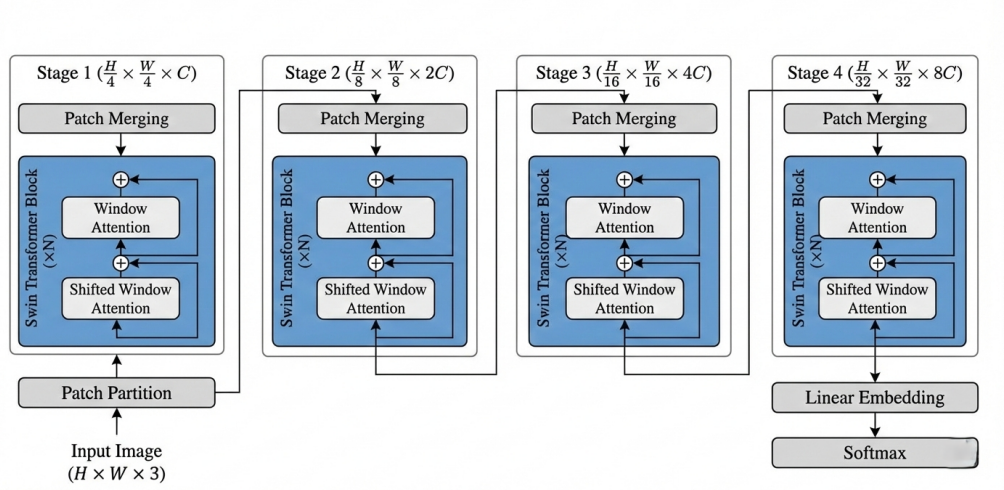

Figure 2.5: Hierarchical Design of Swin Transformer

The main innovation of the Swin Transformer lies in its Shifted Windowing Scheme. Rather than relying on fixed, non overlapping windows in every block, the model alternates between standard and shifted window configurations.Self-attention is first applied within fixed, non overlapping windows, and in the following block, the windows are shifted by a set offset to introduce overlap between adjacent

regions. This overlap enables cross window communication, improving the model's ability to model long range dependencies with minimal additional computational burden.

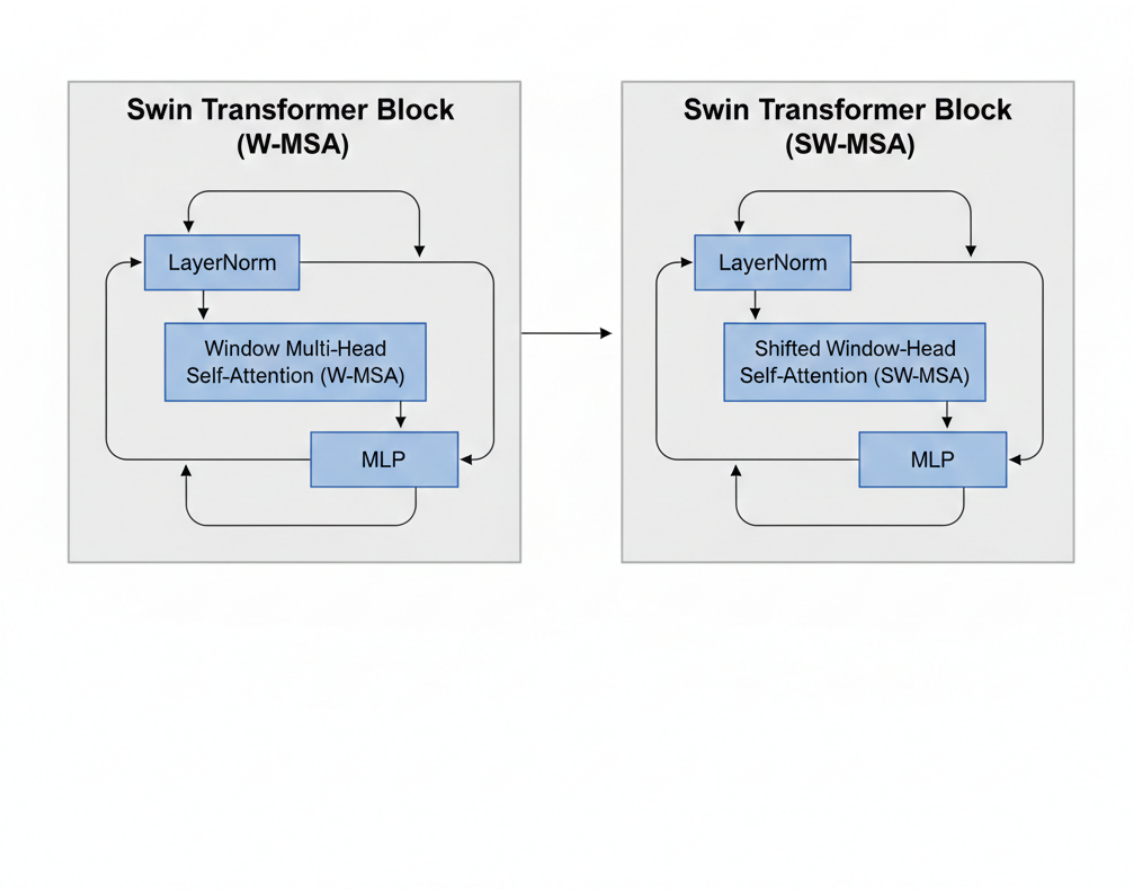

Figure 2.6: Two consequetive Swin blocks. One with the regular window multi-head self attention layer (W-MSA) and the other with a shifted window multi-head self attention layer (SW-MSA).

2.7 EVA

EVA (Exploring the limits of Visual representation at scale)[59] is a vision-centric foundation model designed to systematically investigate the scalability and efficacy of masked visual representation learning. Its architectural core is a vanilla Vision Transformer (ViT), devoid of hierarchical inductive biases or specialized relative positional embeddings, which is deliberately scaled to one billion parameters. This choice reflects an intent to examine the limits of a minimally structured Transformer encoder when coupled with a suitably designed pre-training objective and scaled computational resources.

The operational principle of EVA centers on a masked image modeling (MIM)[60] pretext task. Specifically, the model is trained to reconstruct masked-out, image-text aligned vision features conditioned solely on the visible portions of an input image.

These target features are extracted from a frozen, publicly available CLIP[61] visual encoder, which provides semantically rich supervisory signals derived from contrastive language-image pretraining. Input images are divided into non-overlapping patches, a subset of which, typically 40%, is randomly masked using a block-wise masking strategy. The model processes the visible patches and, via its Transformer encoder layers, generates a contextualized representation for each masked position. A lightweight projection head then maps these representations to the same dimensionality as the CLIP features, and a negative cosine similarity loss is minimized between the predicted and target feature vectors for the masked regions. This formulation constitutes a form of regression-based masked feature prediction, intentionally bypassing intermediate discretization or tokenization steps used in other MIM approaches, thereby simplifying the learning objective.

2.8 BERT

The pursuit of a universal text representation lies at the core of automated natural language processing[62]. A major breakthrough in this domain came with the introduction of pretrained text embeddings such as Word2Vec[63] and GloVe[64]. Bidirectional Encoder Representations from Transformers (BERT)[65] was introduced was introduced in 2019 and became the industry standard in natural language processing.Its core innovation lies in its bidirectionality, achieved through a pre-training objective known as Masked Language Modeling (MLM), which fundamentally distinguishes it from previous unidirectional or shallowly bidirectional models.Architecturally, BERT is built upon the Transformer encoder stack. The model dispenses with recurrent and convolutional layers entirely, relying instead on a multi-layer, multi-head self-attention mechanism to compute contextual representations for every token in an input sequence simultaneously.

The input representation is a sophisticated summation of three embeddings:

1. **Token embeddings**, which are derived from a WordPiece[66] vocabulary.
2. **Segment embeddings**, which distinguish between two sentences in a pairwise task.
3. **Positional embeddings**, which encode the absolute position of each token within the sequence.

A special classification token, [CLS], is prepended to every input sequence. The final hidden state corresponding to this token is used as the aggregate sequence representation for classification tasks. Pairs of sentences are concatenated and separated by a special [SEP] token.

During training, a random subset of input tokens, typically 15%, is replaced with a special [MASK] token. The model is then tasked with predicting the original vocabulary ID of the masked word based solely on its bidirectional context, utilizing the output vector corresponding to the masked position. The second objective is Next Sentence Prediction (NSP), where the model receives pairs of sentences and must classify whether the second sentence logically follows the first in the original document. This task is designed to endow the model with an understanding of inter-sentence relationships, which is beneficial for downstream tasks like question answering and natural language inference.

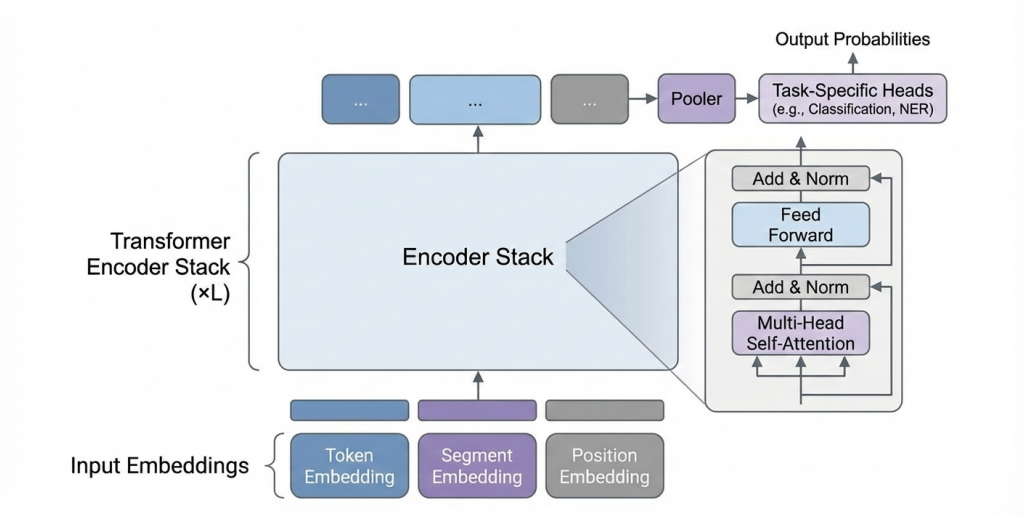

Figure 2.7: An example of BERT's encoding

The resulting pre-trained model captures deep, bidirectional linguistic regularities. For downstream task adaptation, BERT employs a simple yet powerful transfer learning scheme. Minimal task-specific parameters are added, often just a single linear classification layer on top of the pooled [CLS] representation or token level representations. The entire model, including the pre trained parameters, is then fine tuned on labeled data from the target task. This process allows the rich, contextual representations learned during pre training to be efficiently specialized, enabling state-of-the-art performance across a diverse array of natural language processing benchmarks, including GLUE[67], SQuAD[68], and SWAG[69], with minimal architectural modification. BERT thus established a new paradigm for representation learning in NLP, demonstrating that large scale pre training of deep bidirectional models on plain text is a highly effective strategy for achieving robust and generalizable linguistic understanding.

3 Idea and Approach

3.1 Motivation

The current industry standard of Transformer architecture always consists of MLP blocks, this monopoly motivated us to explore alternatives to ensure continued progress. This investigation is motivated by the imperative to explore fundamentally different options beyond this established paradigm. Specifically, the research into replacing MLPs with Kolmogorov-Arnold Networks (KANs) is driven by a compelling theoretical and practical confluence. Theoretically, KANs offer a promising departure from the potentially suboptimal fixed activation functions of MLPs, leveraging learnable activation functions on edges and a structure grounded in the Kolmogorov-Arnold representation theorem. This presents a hypothesis for more parameter-efficient and interpretable function approximation, which could directly address known limitations of Transformers regarding parameter inefficiency and opacity in learned representations. Practically, the Transformer's ubiquity across domains creates a high-impact testbed, even marginal gains in expressivity, efficiency, or interpretability could yield significant downstream benefits. Thus, this work is propelled by the need to rigorously assess this emerging architectural paradigm within a dominant framework, aiming to determine its potential to enhance model performance, reduce computational footprint, and provide deeper insights into the learned algorithmic structures of foundational AI models.

3.2 Problem Statement

Image classification, as a classical research topic in recent years, is one of the core issues of computer vision and the basis of various fields of visual recognition.In general, image classification describes the whole image by manually extracting features or feature learning methods, and then uses the classifier to identify the object category[70]. The core challenge lies in developing models that correctly map high-dimensional, unstructured pixel data into features that a classifier is able to label them accurately. This problem is further compounded by the need for architectures that are not only highly accurate but also computationally tractable for

deployment, data-efficient in learning from limited annotations, and interpretable in their decision making processes. Deep learning approaches, primarily built upon Convolutional Neural Networks (CNNs)[71] and later vision Transformers have achieved great success. However, they often grapple with trade-offs between model complexity and performance, sensitivity to adversarial attacks, and a reliance on vast quantities of labeled data. Therefore, advancing the state of image classification industry requires innovation at the architectural, optimization, and data-representation to create models achieving superior generalization, resilience and efficiency.

The text classification problem represents a cornerstone in Natural Language Processing(NLP), defined by the fundamental challenge of automatically assigning predefined labels or categories to units of textual data. Typical text classification tasks include sentiment analysis, news categorization and topic classification. Recently, researchers show that it is effective to cast many natural language understanding (NLU) tasks[72].A model must effectively construct relevant features and thematic patterns while remaining robust to linguistic phenomena such as ambiguity, irony, domain-specific language, and the inherent high-dimensional sparsity of text representations. While modern deep learning approaches, from recurrent neural networks and Convolutional Neural Networks to large language models (LLMs), have advanced the overall performance, they come with their own set of challenges.These include the substantial computational and data requirements for training, the risk of learning social biases present in training corpora, and the frequent opacity of model reasoning. In consequence, advancing text classification demands not only optimizing the models accuracy benchmarks but also pushing forward domains enabling for a more reliable, scalable and efficient text analysis.

4 Methodology

In this section, we are presenting the practical implementation of B-spline KAN and GR-KAN on pretrained Transformers, comparing them with the traditional MLP architectures. We used benchmark models in computer vision for image classification tasks such as ViT, Swin and EVA, as well as BERT for text classification. The implementation process includes data preprocessing, model design, training procedures and evaluation techniques. Specifically, the models where trained and evaluated using stratified 5-fold cross validation, to ensure the same data distribution on all of the classes, on a 80%-20% split. The metrics we used were Cross Entropy Loss, Multi Class Accuracy and training time per fold to measure their performance. Each model was fine-tuned for 5-6 epochs per fold.

The key model architectures we identified were:

- Replace only the classification head of the model with KAN.
- Half of the model with MLPs and half with KANs.
- Replace all of the MLPs of the model with KANs.

The hyper-parameters chosen the B-spline KAN were grid-size $g = 2$ and spline order $k = 2$ in order to minimize the exponential growth of parameters in Transformers, since Transformers have already high capacity. GR-KAN does not increase the number of parameters of the model exponentially since it is using the grouping parameter. The hyper-parameters chosen for GR-KAN were number of groups $g = 8$ the order of the polynomials, $m = 5$ for the polynomial in the enumerator and $n = 4$ for the denominator. Various combinations of GR-KAN hyper-parameters were evaluated, but the model's performance consistently degraded.

We experimented with Adam and AdamW [73] optimizers. All in all, AdamW being more robust since it adds the weight decay regularization on of the mix and the models were able to generalize better. All experiments were conducted using Pytorch lightning as the primary framework, using the the single GPU provided by Kaggle.

4.1 Datasets

The datasets used were:

1. **Skyview dataset**[74]:Skyview is a curated dataset for aerial landscape classification featuring 15 diverse categories, each comprising 800 high-quality images at a resolution of 256x256 pixels. This dataset is a fusion of images sourced from the publicly available AID and NWPU-Resisc45 datasets. The compilation is designed to facilitate research and development in the field of computer vision, particularly in the context of aerial landscape analysis. The dataset is balanced, each class contains 800 images.

2. **Suicide Detection dataset**[75]:The dataset is a collection of posts from the "SuicideWatch" and "depression" subreddits of the Reddit platform. The posts are collected using Pushshift API. All posts that were made to "SuicideWatch" from Dec 16, 2008(creation) till Jan 2, 2021, were collected while "depression" posts were collected from Jan 1, 2009, to Jan 2, 2021. All posts collected from SuicideWatch are labeled as suicide, While posts collected from the depression subreddit are labeled as depression. Non-suicide posts are collected from r/teenagers.

   The dataset is consisted of 232073 rows and 2 columns, a column for the text that is going to be classified and a label. The dataset is balanced with each class having 116037 data points.

3. **Food classification dataset**[76]:The "Food Image Classification Dataset" features 24K unique images from Google resources, focusing on 34 varieties of Indian and Western appetizers. This dataset enables accurate classification, making it ideal for menu recommendation and inventory management, while exploring the rich culinary world.

   The dataset is imabalanced, consisting mode images of widely known foods like hot dogs and donuts while there are less images of less known foods such as samosa.

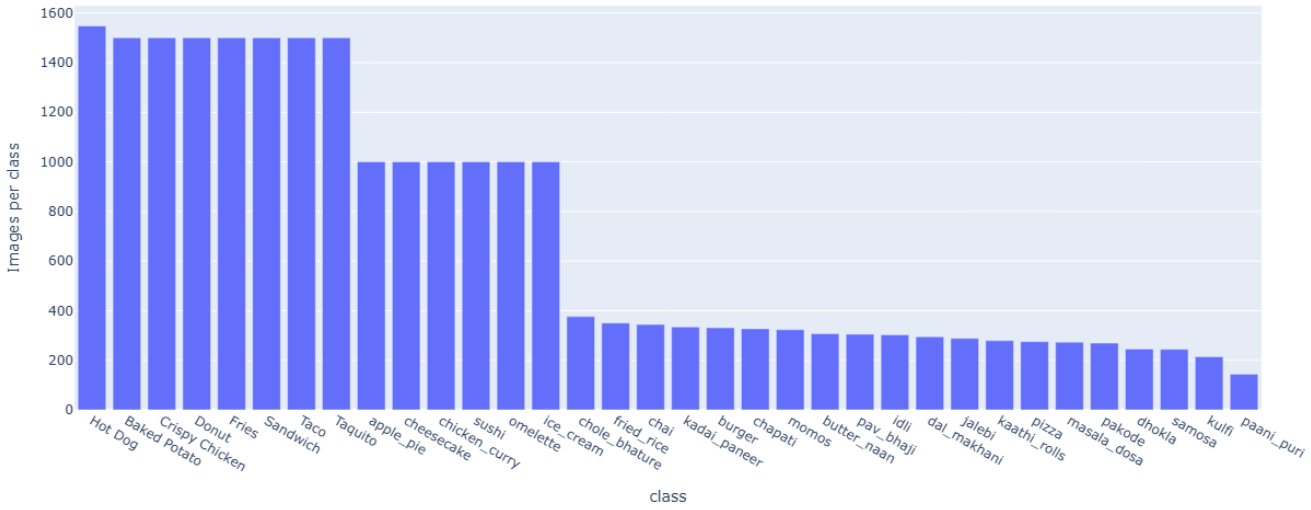

Figure 4.1: The class distribution of the Food dataset

At the same time, the resolution of the images varies a lot. Images have a minimum (width,height) starting from (100,94) and can go up to (7599,6000). The mean of (width,height) is (481.095,426.133) with a standard deviation of (494.156,413.593).

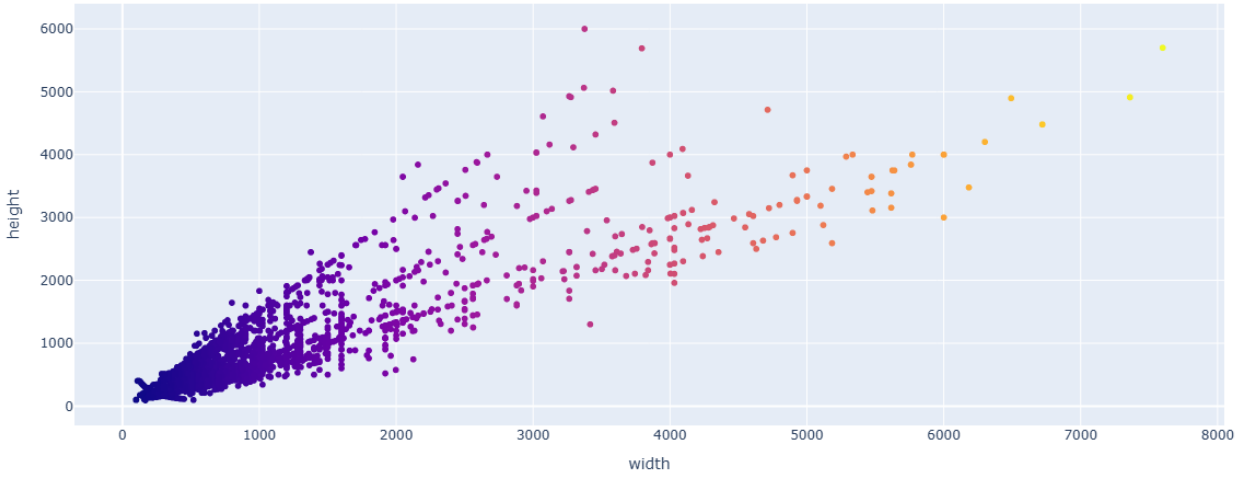

Figure 4.2: A scatter plot of the image resolutions of the Food dataset

4.2 Model architecture

This section describes the architectural configurations and implementation details of the models employed to evaluate performance on the selected datasets. For the

Vision Transformer (ViT), we initially used a pretrained model containing approximately 5.7 million parameters on the SkyView dataset. This model was pretrained on ImageNet-21k[77] and subsequently fine-tuned on ImageNet-1k. However, its performance on the food image dataset was suboptimal. To address this limitation, we increased the dimensionality of the linear layers, thereby expanding the model capacity to approximately 30 million parameters. While this modification improved classification performance, it came at the cost of increased training time.

For the Swin Transformer, we employed a pretrained SwinV2[78] model with 28.3 million parameters for both image datasets. This model was pretrained on ImageNet-21k and leverages a hierarchical architecture with shifted window attention, enabling efficient modeling of multi-scale visual features.

For the EVA Transformer, we utilized a pretrained EVA-02 feature representation model. This model was trained on ImageNet-22k using masked image modeling, with EVA-CLIP serving as the teacher network. The EVA-02 architecture is based on Vision Transformers and incorporates mean pooling, SwiGLU[79] activation functions, Rotary Position Embeddings (RoPE)[80], and additional Layer Normalization layers within the MLP blocks (for the Base and Large variants), all of which contribute to improved representation learning and training stability.

For the text-based experiments, we employed a pretrained BERT uncased model with approximately 4 million parameters. BERT is a Transformer-based language model that utilizes a bidirectional self-attention mechanism to learn contextualized word representations from large-scale text corpora. The uncased variant processes all input text in lowercase, effectively reducing vocabulary size and mitigating sensitivity to capitalization, which can be beneficial for tasks where case information is not critical. The model was pretrained using a combination of masked language modeling and next sentence prediction objectives on large datasets such as BooksCorpus[81] and English Wikipedia. Owing to its strong contextual representation capabilities, the BERT uncased model serves as a robust backbone for a wide range of downstream natural language processing tasks after fine-tuning.

For the computer vision Transformers we chose a starting learning rate $lr = 10^{-4}$ and use the StepLR scheduler to decrease the learning rate by 0.1 every 3 epochs. While a learning rate $lr = 2 * 10^{-5}$ was chosen for BERT.

Additionally, the following techniques were used for the computer vision models:

- **Dropout**:Is a regularization technique. It prevents overfitting and provides a way of approximately combining exponentially many different neural network architectures efficiently. The term “dropout” refers to dropping out units (hidden and visible) in a neural network. By dropping unit out, we mean temporarily removing it from the network, along with all its incoming and out-

going connections [82].In the simplest case, each unit is retained with a fixed probability p independent of other units, where p is another hyperparameter that can be chosen through hyperparameter tuning or empirically.

- **Layer Normalization**: A simple normalization method to improve the training speed for various neural network models. Unlike batch normalization, this method directly estimates the normalization statistics from the summed inputs to the neurons within a hidden layer so the normalization does not introduce any new dependencies between training cases.The mean and the standard deviation of the layer normalization are calculated by the following formulas:

$$\mu = \frac{1}{H}\sum_{i=1}^{H} a_i \tag{4.1}$$

$$\sigma = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(a_i - \mu_i)^2} \tag{4.2}$$

  Where H is number of hidden units in a layer[83].

- **Decoupled Weight decay**[84]: A regularization technique used in neural network optimization where weight decay is applied independently of the gradient based update, rather than being mixed into the loss gradient. It was introduced to address shortcomings of traditional L2 regularization when used with adaptive optimizers such as Adam. Empirically it has been shown, that the version of Adam with decoupled weight decay (AdamW) yields substantially better generalization performance than the common implementation of Adam with L2 regularization. This technique helps the neural networks learn smoother, simpler functions which most of the time generalize better to spiky, noisy ones. It has been observed that lower weight decay values help models prevent underfitting while higher weight decay values prevent overfitting.

4.3 Data Preprocessing

In computer vision, in order to achieve the best possible performance, scientists apply image augmentations to the image data. These techniques simulate real world scenarios, where the images are flipped, cropped, have different lighting, colors and size. In this way, the models learn to generalize better and make better predictions.

In this thesis the following image augmentations were used:

- Random Horizontal Flip: Randomly flips the image horizontally to augment left/right symmetry.
- Random rotation: Randomly rotates the image by a certain degree range, which helps recognize the existing objects in the image regardless of their orientations.
- Color Jitter: Randomly adjusts brightness, contrast and saturation to simulate varying lighting conditions.
- Rand Augment:Randomly applies a set of transformations in a sequential manner with a specified magnitude (reduced search space and computational cost compared to Auto Augment).

Real world textual data, on the other hand, often contain noise such as irregular spacing, encoding artifacts, or punctuation and emoticons, which can introduce variability unrelated to the underlying semantic content. Preprocessing mitigates these issues, allowing the model to focus on linguistically meaningful patterns rather than spurious surface level variations. Furthermore, preprocessing plays an important role in managing input sequence length, as many documents exceed BERT's fixed maximum token limit. By cleaning, truncating, or segmenting text in a controlled manner, researchers can ensure efficient use of the model's context window while maintaining task-relevant information.

Text preprocessing is not a fixed sequence of steps that everybody follows. It is related to the dataset and task we are dealing with. Sometimes, over processing text data might lead to missing keywords that would be beneficial on the classification of a sentence for example. That means, we have to explore the data and the task first and the process them accordingly.

In this thesis we removed special punctuation symbols like explanation marks, semicolons, colons and parenthesis , unicode and special character like emojis and currency/mathematical symbols. All these characters that were removed, were replaced by an empty string.

4.4 KAN configuration

The Kolmogorov–Arnold Network (KAN) replaces convolutional matrix multiplications with edge wise spline-based transformations. In this formulation, each connection is represented by a univariate cubic spline with fixed knot locations and trainable control point coefficients. Inputs are processed independently through these spline functions, and each output unit aggregates the transformed inputs via summation, in accordance with the Kolmogorov–Arnold representation theorem.

To ensure a fair comparison with multilayer perceptrons (MLPs), the input and output dimensionalities of each KAN layer were matched to those of the corresponding linear layer it replaced. Unlike MLPs, where increasing network width results in a larger number of scalar weight parameters and dense matrix operations, the capacity of a KAN scales with the number of spline functions. In our experiments, B-splines of order two were employed, with two grid points per input dimension, and a GELU base activation was used to maintain consistency with the activation functions adopted in the Transformer architecture.

4.5 GR-KAN configuration

GR-KAN is a KAN variant which replaces the B-splines with rational polynomial functions and groups them in order to reduce the number of parameters and computation.To optimize the evaluation of polynomials, Horner's method[85] is employed, which reformulates a polynomial in a nested form to reduce the computation. Another goal of this approach is to make the rational functions CUDA [86] friendly and implement it efficiently on parallelizable devices like GPU.

In the same manner as vanilla KANs we chose to have the to configure GR-KAN layers with same input and output features as the corresponding linear layers they were replacing, in order for the comparison to be fair. Additionally, we chose to polynomials of order $m = 5, n = 4$ for the polynomials on the enumerator and denominator respectively, as suggested on the paper, and a number of groups $g = 8$, in order to reduce the exponential growth of parameters. The lower the number of groups the closer means that more and more unique rational functions have to be learned, thus increasing the parameter count. When the number of groups is 0, this approach resembles a vanilla KAN, but with rational functions instead of B-splines, because a unique function has to be learnt for each edge.

4.6 Comperative Experiments and Results

The following tables summarize our experiments with the architectures and evaluation techniques explained.

4.6.1 Skyview dataset

Table 4.1: Swin on the Skyview dataset

Strategy	Average Loss	Average Accuracy	Training time/fold
Swin	0.078	0.979	588s
Swin-KAT head	2,61	0.7621	446s
Swin-KAT 1st layer	0.1	0.969	644s
Swin-KAT full	1.37	0.52	1048s
Swin-KAN head	0.09	0.973	589s

Table 4.2: ViT on the Skyview dataset

Strategy	Average Loss	Average Accuracy	Training time/fold
ViT	2,66	0.626	318s
ViT-KAN head	0.469	0.856	337s
ViT-KAT head	0.392	0.875	489s
ViT-KAN 5 blocks	0.42	0.865	924s
ViT-KAT 5 blocks	0.343	0.893	680s

Table 4.3: EvA on the skyview dataset

Strategy	Average Loss	Average Accuracy	Training time/fold
EvA	0.531	0.837	637s
EvA-KAN head	0.213	0.934	637s
EvA-KAT head	0.621	0.812	638s
EvA-KAT 5blocks	1.3	0.576	820s
EVA-KAN 5 blocks	0.232	0.929	1349s

4.6.2 Food dataset

Table 4.4: ViT on the food dataset

Strategy	Average Loss	Average Accuracy	Training time/fold
ViT	2,15	0.359	1217s
ViT-KAN head	2.24	0.345	1019s
ViT-KAN 5 blocks	2.25	0.32	924s
ViT-KAT head	2.41	0.287	1210s

Table 4.5: Swin on the food dataset

Strategy	Average Loss	Average Accuracy	Training time/fold
Swin	0.32	0.918	1411s
Swin-KAT head	3.25	0.06	1419s
Swin-KAT 2 layers	2.49	0.19	2109s
Swin-KAN head	0.307	0.916	1415s

Table 4.6: EvA on the food dataset

Strategy	Average Loss	Average Accuracy	Training time/fold
EvA	2.51	0.243	1500s
EvA-KAN head	0.538	0.839	1504s
EvA-KAN 5 blocks	0.539	0.835	3191s
EvA-KAT head	1.84	0.444	1504s
EvA-KAT 5blocks	2.43	0.3	1926s

4.6.3 Suicide Detection Dataset

Table 4.7: BERT on the text dataset

Strategy	Average Loss	Average Accuracy	Training time/fold
BERT	0.176	0.944	332s
BERT-KAN 1 layer	0.181	0.943	413s
BERT-KAT 1 layer	0.172	0.944	414s
BERT-KAT 2 layers	0.166	0.945	503s
BERT-KAN 2 layers	0.176	0.943	440s

Across the Skyview dataset (Tables 4.1-4.3), replacing only the classification head of the model with KAN and GR-KAN layers generally leads to overall accuracy improvements over the baseline models, notably SWIN and EVA, where head level substiutions achieve same or higher accuracy on comparable training time. In contrast, extending KAN or GR-KAN replacements deeper into the network, such as across multiple Transformer blocks or the full MLP stack, often results in diminishing returns or performance degradation, accompanied by substantially increased training time. This trend is particularly evident in the Swin-KAT and EVA-KAT configurations, where full or multi block replacements yield lower accuracy despite higher computational cost. Similar patterns appear in the Food dataset too (Tables 4.4-4.6) where KAN heads improve performace, especially to EVA, while deeper substitutions diminish performance as the training time per fold increases at the same time. Finally, the text classification results on the Suicide Detection dataset (Table 4.7) indicate that both KAN and GR-KAN variants achieve performance comparable to standard MLPs in BERT, with marginal differences in accuracy but noticeable increases in training time. Overall, the tabulated results suggest that KAN and GR-KAN layers are most effective when applied selectively and should be considered as an alternative when fine tuning Transformers.

4.7 Expainability

Despite the remarkable empirical success of deep learning models across a wide range of tasks, their increasing complexity has led to a growing concern regarding transparency and trustworthiness. Modern neural networks, particularly large-scale architectures, are often regarded as black-box systems, offering limited insight into the reasoning behind their predictions. This lack of interpretability poses challenges for model validation, debugging, and deployment in high-stakes domains. Consequently, explainability has emerged as a critical area of research, aiming to develop methods that provide human-understandable explanations of model behavior and decision making processes.

In this section we aim to present the explainability of KANs as it is their highlighted advantage over MLPs.

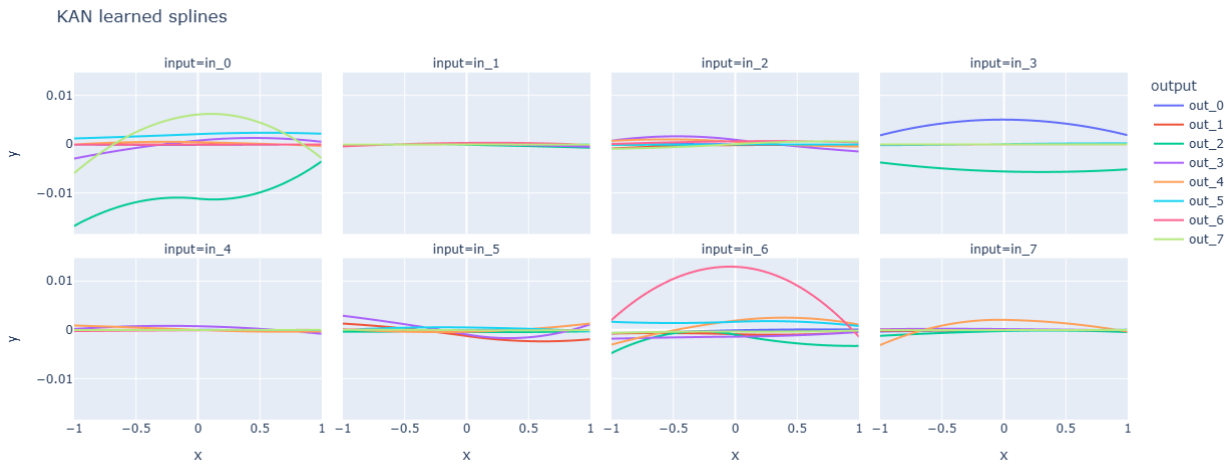

Figure 4.3: The splines learnt from EVA-KAN head for 8 input and output features on the Skyview dataset

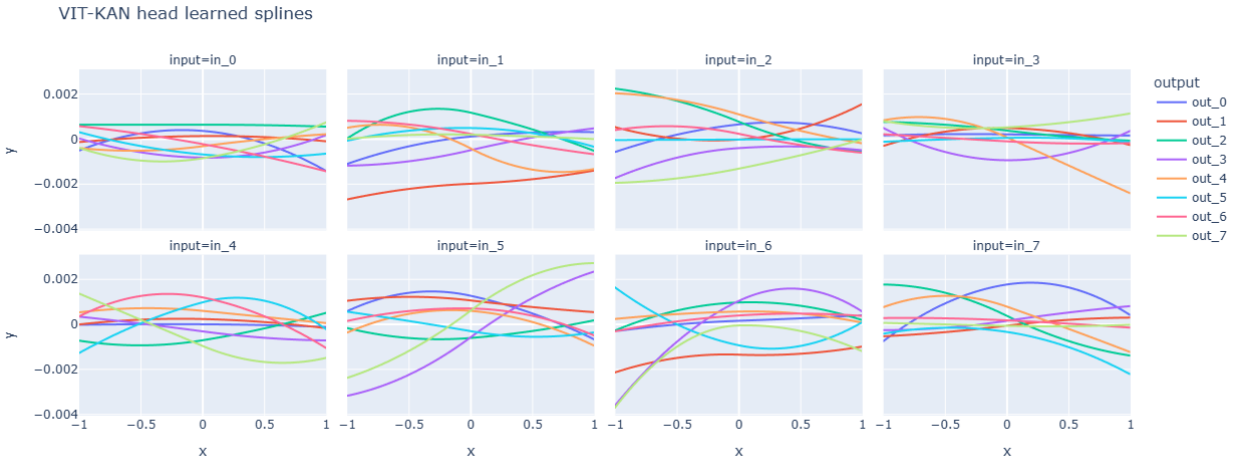

Figure 4.4: The splines learnt from VIT-KAN head for 8 input and output features on the Skyview dataset

The learnt spline functions of the VIT-KAN head exhibit stable and smooth behaviour across all input dimensions, showing small-scale, slow-moving fluctuations, indicating that KAN heads act as non linear adaptors on top of VIT backbone features. Compared to the EVA-KAN case showcased in Figure 4.3, the VIT-KAN splines display generally smaller magnitudes across input channels, suggesting weaker dependence upon sophisticated input specific nonlinearities. While certain dimensions, specifically inputs 2, 5, and 6, continue to manifest significant curvature and variation across outputs, the degree of individual input specialization is less intensive than in the EVA–KAN model. Whereas EVA–KAN frequently exhibits dominant and exceptionally expressive spline dynamics for certain inputs, the current model demonstrates a more distributed or moderated functional response. This implies that the EVA backbone provides feature representations that benefit more from localized nonlinear refinement at the head, whereas the ViT backbone appears to deliver features that are closer to linearly separable, reducing the demand for expressive spline-based transformations. In summary, The EVA-KAN model demonstrates superior characteristics for a KAN architecture. It is functioning exactly as KANs are theoretically supposed to: it is pruning irrelevant features (via sparsity) and learning strong, interpretable non-linear functions for the relevant ones. In contrast, the VIT-KAN appears to be struggling to leverage the KAN's unique properties. It has defaulted to a "dense" behavior similar to a standard MLP, where small weights are distributed across all connections, failing to isolate specific meaningful features.

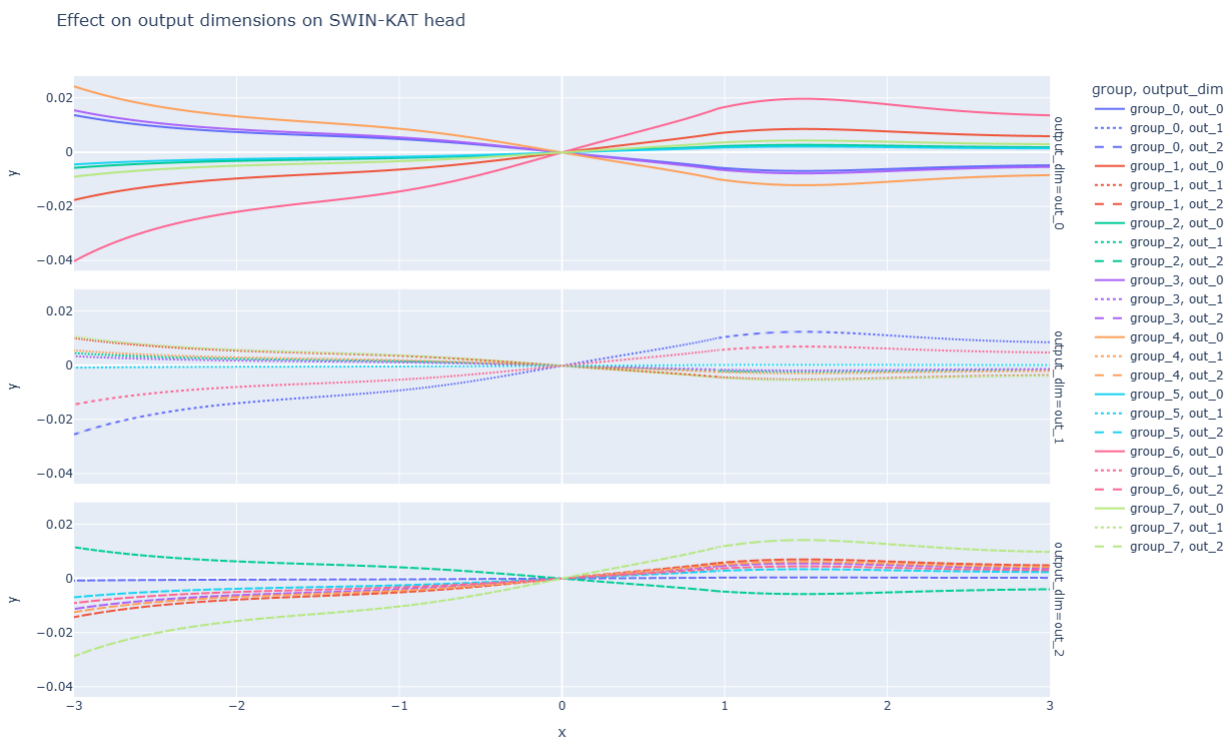

Figure 4.5: The learnt rational functions for each group on the SWIN-KAT head model for 3 output dimensions

Empirical visualization of the group wise rational activation functions within the SWIN-KAT head validates the operational efficacy and numerical stability of the GR-KAN layer. The learned mappings across all groups are smooth and well conditioned, devoid of pathological oscillations or asymptotes, which suggests robust regularization and stable training dynamics. The emergence of heterogeneous nonlinear responses confirms that the grouping mechanism effectively mitigates feature redundancy. Furthermore, the strategic reuse of these nonlinearities through distinct linear recombinations facilitates efficient feature sharing and task specific specialization. The observed transition from near linear behavior at the origin to smooth nonlinear deviations at higher magnitudes aligns with the expected dynamics of normalized feature distributions in Transformer architectures. Consequently, the GR-KAN head serves as a transparent and expressive alternative to conventional MLP-based configurations. It can be observed that the activation function in group 0 always resembles a linear function, for group 1 the function resembles a tanh/sigmoid function and for the rest of the activation functions they resemble a complex rational function.

5 Conclusions

The main conclusion of this thesis is that Kolmogorov-Arnold Networks, while theoretically elegant and powerful in certain settings, cannot serve as a direct replacement to multi-layer perceptrons within Transformer architectures. Across a set of vision and language models, the results show that KANs and GR-KANs can match or improve performance in specific configurations, particularly when used as classifier heads or limited portions of the network. However, these gains are accompanied by large increases in training time and computational cost.

In large scale vision models, such as Swin and EvA, KAN based heads demonstrate notable performance improvements on certain datasets. However, replacing higher level MLP blocks tend to degrade performance or introduce instability, highlighting scalability and optimization challenges. In text classification with BERT, KANs and GR-KANs both achieve comparable results and their difference with MLPs is negligible, reinforcing the idea that their benefits are task dependent and not architecture dependent.

Overall, this thesis provides empirical evidence that KANs should be viewed not as a direct replacement to MLPs but as another available asset that everyone can experiment on. It demonstrates that while KANs are unlikely to replace MLPs as a whole in Transformers, they remain a promising research direction when applied selectively and with architectural awareness.

5.1 Limitations

A primary limitation this study was the available hardware we had and its reliance to the GPUs provided by Kaggle. This constraint imposed strict limits on the scope of this thesis as it restricted us to fine tune the models for 5-6 epochs per fold which may not have been sufficient for KANs to fully converge, especially when we replaced them with MLPs deeper in the model's architecture. It also limited us from using deeper neural networks. This is particularly relevant for the full KAN replacement, where the parameter count grows significantly large.

Due to computationally intensive nature of KANs and the hardware limits, hyperparameter optimization was not feasible. This study chose some fixed hy-

perparameters in order to minimize the parameter explosion rather than optimal performance. It remains unknown if a combination of different hyperparameters would allow KANs to outperform MLPs

The experiments showed that replacing all MLP blocks often led to severe accuracy drops (Swin-KAT full dropping to 0.52 accuracy). This limitation suggests that the current weight initialization strategies or normalization techniques (Layer Normalization) designed for MLPs are not directly transferable to deep KAN architectures, preventing a fair assessment of a complete KAN-Transformer.

5.2 Future Work

The primary drawback to KAN adoption is training latency. Future work should include developing CUDA optimized kernels for B-splines and parallelizable training similar to how matrix multiplications are optimized in MLPs, is essential to make KANs competitive in training speed.

The instability observed when replacing higher level MLPs suggests that standard initialization techniques like Xavier[87] are not suited for the spline weights. Future research should introduce new initialization techniques and investigate how layer normalization interacts with the learnable activation functions. This way training could be more stable.

Given that KANs excelled as classification heads but straggled as backbone replacements, a hybrid approach is in need of investigation. Future architectures could employ MLPs in early blocks, extracting the simpler features, for their speed and stability, while utilizing KANs in the deeper layers in order to capture complex, non-linear relationships and improve interpretability.

The experiments in this thesis were conducted in mid-sized datasets, due to limited available GPU resources. As these models grow in size, they often begin to display surprising abilities that weren't intentionally programmed, testing KAN-Transformers on large scale datasets like ImageNet-1k or large text corpora would help help determine if the inductive biases of KANs provide greater benefits when data is abundant. Finally, due to the Transformer properties, training a full KAN layered Transformer on more epochs (usually 50+) should be tested.

References

1. Graves, A. & Schmidhuber, J. Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural networks* **18,** 602–610 (2005).
2. Cho, K. *et al.* Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078* (2014).
3. Li, Z., Liu, F., Yang, W., Peng, S. & Zhou, J. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE transactions on neural networks and learning systems* **33,** 6999–7019 (2021).
4. Islam, S. *et al.* A comprehensive survey on applications of transformers for deep learning tasks. *Expert Systems with Applications* **241,** 122666 (2024).
5. Sutskever, I., Vinyals, O. & Le, Q. V. Sequence to sequence learning with neural networks. *Advances in neural information processing systems* **27** (2014).
6. Rosenblatt, F. *The perceptron, a perceiving and recognizing automaton:(Project Para)* (Cornell Aeronautical Laboratory, 1957).
7. Rosenblatt, F. *et al. Principles of neurodynamics: Perceptrons and the theory of brain mechanisms* (Spartan books Washington, DC, 1962).
8. Hornik, K., Stinchcombe, M. & White, H. Multilayer feedforward networks are universal approximators. *Neural networks* **2,** 359–366 (1989).
9. Delashmit, W. H., Manry, M. T., *et al. Recent developments in multilayer perceptron neural networks* in *Proceedings of the seventh annual memphis area engineering and science conference, MAESC* **7** (2005), 33.
10. Yang, X. & Wang, X. Kolmogorov-arnold transformer. *arXiv preprint arXiv:2409.10594* (2024).
11. Cunningham, H., Ewart, A., Riggs, L., Huben, R. & Sharkey, L. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600* (2023).

12. Kolmogorov, A. N. *On the representations of continuous functions of many variables by superposition of continuous functions of one variable and addition* in *Dokl. Akad. Nauk USSR* **114** (1957), 953–956.

13. Kolmogorov, A. N. *On the representation of continuous functions of several variables by superpositions of continuous functions of a smaller number of variables* (American Mathematical Society, 1961).

14. Liu, Z. *et al.* Kan: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756* (2024).

15. Ji, T., Hou, Y. & Zhang, D. A comprehensive survey on kolmogorov arnold networks (kan). *arXiv preprint arXiv:2407.11075* (2024).

16. Vaca-Rubio, C. J., Blanco, L., Pereira, R. & Caus, M. Kolmogorov-arnold networks (kans) for time series analysis. *arXiv preprint arXiv:2405.08790* (2024).

17. Poeta, E., Giobergia, F., Pastor, E., Cerquitelli, T. & Baralis, E. *A benchmarking study of kolmogorov-arnold networks on tabular data* in *2024 IEEE 18th International Conference on Application of Information and Communication Technologies (AICT)* (2024), 1–6.

18. Bodner, A. D., Tepsich, A. S., Spolski, J. N. & Pourteau, S. Convolutional kolmogorov-arnold networks. *arXiv preprint arXiv:2406.13155* (2024).

19. Mohan, K., Wang, H. & Zhu, X. Kans for computer vision: An experimental study. *arXiv preprint arXiv:2411.18224* (2024).

20. Bengio, Y. *et al.* Learning deep architectures for AI. *Foundations and trends® in Machine Learning* **2,** 1–127 (2009).

21. Deng, L., Yu, D., *et al.* Deep learning: methods and applications. *Foundations and trends® in signal processing* **7,** 197–387 (2014).

22. Utgoff, P. E. & Stracuzzi, D. J. Many-layered learning. *Neural computation* **14,** 2497–2529 (2002).

23. Glorot, X., Bordes, A. & Bengio, Y. *Deep sparse rectifier neural networks* in *Proceedings of the fourteenth international conference on artificial intelligence and statistics* (2011), 315–323.

24. Bengio, Y., Lamblin, P., Popovici, D. & Larochelle, H. Greedy layer-wise training of deep networks. *Advances in neural information processing systems* **19** (2006).

25. Bengio, Y., Lecun, Y. & Hinton, G. Deep learning for AI. *Communications of the ACM* **64,** 58–65 (2021).

26. Hartmann, C. & Richter, L. *Transgressing the boundaries: towards a rigorous understanding of deep learning and its (non-)robustness* 2023. arXiv: 2307.02454 [cs.LG]. https://arxiv.org/abs/2307.02454.

27. Belkin, M. *Fit without fear: remarkable mathematical phenomena of deep learning through the prism of interpolation* 2021. arXiv: 2105.14368 [stat.ML]. https://arxiv.org/abs/2105.14368.

28. Vilone, G. & Longo, L. Explainable artificial intelligence: a systematic review. *arXiv preprint arXiv:2006.00093.* https://arxiv.org/abs/2006.00093 (2020).

29. Bartoldson, B. R., Kailkhura, B. & Blalock, D. *Compute-Efficient Deep Learning: Algorithmic Trends and Opportunities* 2023. arXiv: 2210.06640 [cs.LG]. https://arxiv.org/abs/2210.06640.

30. Kariri, E., Louati, H., Louati, A. & Masmoudi, F. Exploring the advancements and future research directions of artificial neural networks: a text mining approach. *Applied Sciences* **13,** 3186 (2023).

31. Mienye, I. D. & Swart, T. G. A comprehensive review of deep learning: Architectures, recent advances, and applications. *Information* **15,** 755 (2024).

32. Vaswani, A. *et al.* Attention is all you need. *Advances in neural information processing systems* **30** (2017).

33. He, K., Zhang, X., Ren, S. & Sun, J. *Deep residual learning for image recognition* in *Proceedings of the IEEE conference on computer vision and pattern recognition* (2016), 770–778.

34. Lin, T., Wang, Y., Liu, X. & Qiu, X. A survey of transformers. *AI open* **3,** 111–132 (2022).

35. Touvron, H. *et al. Training data-efficient image transformers distillation through attention* 2021. arXiv: 2012.12877 [cs.CV]. https://arxiv.org/abs/2012.12877.

36. Lins, A. & Ludermir, T. B. *Hybrid optimization algorithm for the definition of MLP neural network architectures and weights* in *Fifth International Conference on Hybrid Intelligent Systems (HIS'05)* (2005), 6–pp.

37. Riedmiller, M. & Lernen, A. Multi layer perceptron. *Machine learning lab special lecture, University of Freiburg* **24,** 11–60 (2014).

38. Augustine, M. T. A survey on universal approximation theorems. *arXiv preprint arXiv:2407.12895* (2024).

39. Popescu, M.-C., Balas, V. E., Perescu-Popescu, L. & Mastorakis, N. Multilayer perceptron and neural networks. *WSEAS Transactions on Circuits and Systems* **8,** 579–588 (2009).

40. Rumelhart, D. E., Hinton, G. E. & Williams, R. J. Learning representations by back-propagating errors. *nature* **323,** 533–536 (1986).

41. Nair, V. & Hinton, G. E. *Rectified linear units improve restricted boltzmann machines* in *Proceedings of the 27th international conference on machine learning (ICML-10)* (2010), 807–814.

42. Bengio, Y., Simard, P. & Frasconi, P. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks* **5,** 157–166 (1994).

43. Ramachandran, P., Zoph, B. & Le, Q. V. Searching for activation functions. *arXiv preprint arXiv:1710.05941* (2017).

44. Hendrycks, D. Gaussian Error Linear Units (Gelus). *arXiv preprint arXiv:1606.08415* (2016).

45. Kilani, B. H. Kolmogorov-arnold networks: Key developments and uses. *Qeios* (2024).

46. Cang, Y., Shi, L., *et al.* Can kan work? exploring the potential of kolmogorov-arnold networks in computer vision. *arXiv preprint arXiv:2411.06727* (2024).

47. Wang, Y. *et al.* Kolmogorov–Arnold-Informed neural network: A physics-informed deep learning framework for solving forward and inverse problems based on Kolmogorov–Arnold Networks. *Computer Methods in Applied Mechanics and Engineering* **433,** 117518 (2025).

48. Mallick, S., Ghosh, S. & Roy, T. KAN-Therm: A lightweight battery thermal model using Kolmogorov-Arnold Network. *arXiv preprint arXiv:2509.09145* (2025).

49. Perperoglou, A., Sauerbrei, W., Abrahamowicz, M. & Schmid, M. A review of spline function procedures in R. *BMC medical research methodology* **19,** 46 (2019).

50. Eilers, P. H. & Marx, B. D. Flexible smoothing with B-splines and penalties. *Statistical science* **11,** 89–121 (1996).

51. Boullé, N., Nakatsukasa, Y. & Townsend, A. Rational neural networks. *Advances in neural information processing systems* **33,** 14243–14253 (2020).

52. Walsh, J. L. *Interpolation and approximation by rational functions in the complex domain* (American Mathematical Soc., 1935).

53. Baker Jr, G. A. & Gammel, J. L. The padé approximant. *Journal of Mathematical Analysis and Applications* **2,** 21–30 (1961).

54. Molina, A., Schramowski, P. & Kersting, K. Pad\'e activation units: End-to-end learning of flexible activation functions in deep networks. *arXiv preprint arXiv:1907.06732* (2019).

55. Dosovitskiy, A. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

56. Han, K. *et al.* A survey on visual transformer. *arXiv preprint arXiv:2012.12556* (2020).

57. Yuan, L. *et al. Tokens-to-token vit: Training vision transformers from scratch on imagenet* in *Proceedings of the IEEE/CVF international conference on computer vision* (2021), 558–567.

58. Liu, Z. *et al. Swin transformer: Hierarchical vision transformer using shifted windows* in *Proceedings of the IEEE/CVF international conference on computer vision* (2021), 10012–10022.

59. Fang, Y. *et al. Eva: Exploring the limits of masked visual representation learning at scale* in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2023), 19358–19369.

60. Chen, M. *et al. Generative pretraining from pixels* in *International conference on machine learning* (2020), 1691–1703.

61. Radford, A. *et al. Learning transferable visual models from natural language supervision* in *International conference on machine learning* (2021), 8748–8763.

62. Koroteev, M. V. BERT: a review of applications in natural language processing and understanding. *arXiv preprint arXiv:2103.11943* (2021).

63. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S. & Dean, J. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* **26** (2013).

64. Pennington, J., Socher, R. & Manning, C. D. *Glove: Global vectors for word representation* in *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (2014), 1532–1543.

65. Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. *Bert: Pre-training of deep bidirectional transformers for language understanding* in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)* (2019), 4171–4186.

66. Song, X., Salcianu, A., Song, Y., Dopson, D. & Zhou, D. *Fast wordpiece tokenization* in *Proceedings of the 2021 conference on empirical methods in natural language processing* (2021), 2089–2103.

67. Wang, A. *et al. GLUE: A multi-task benchmark and analysis platform for natural language understanding* in *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP* (2018), 353–355.

68. Rajpurkar, P., Zhang, J., Lopyrev, K. & Liang, P. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250* (2016).

69. Zellers, R., Bisk, Y., Schwartz, R. & Choi, Y. Swag: A large-scale adversarial dataset for grounded commonsense inference. *arXiv preprint arXiv:1808.05326* (2018).

70. Chen, L. *et al.* Review of image classification algorithms based on convolutional neural networks. *Remote Sensing* **13,** 4712 (2021).

71. LeCun, Y. *et al.* Backpropagation applied to handwritten zip code recognition. *Neural computation* **1,** 541–551 (1989).

72. Minaee, S. *et al.* Deep learning–based text classification: a comprehensive review. *ACM computing surveys (CSUR)* **54,** 1–40 (2021).

73. Adam, K. D. B. J. *et al.* A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* **1412** (2014).

74. *Skyview dataset* https://www.kaggle.com/datasets/ankit1743/skyview-an-aerial-landscape-dataset.

75. *Suicide and depression detection dataset* https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch.

76. *Food image classification dataset* https:https://www.kaggle.com/datasets/harishkumardatalab/food-image-classification-dataset.

77. Ridnik, T., Ben-Baruch, E., Noy, A. & Zelnik-Manor, L. Imagenet-21k pretraining for the masses. *arXiv preprint arXiv:2104.10972* (2021).

78. Liu, Z. *et al. Swin transformer v2: Scaling up capacity and resolution* in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (2022), 12009–12019.

79. Shazeer, N. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202* (2020).

80. Su, J. *et al.* Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* **568,** 127063 (2024).

81. Bandy, J. & Vincent, N. *Addressing "Documentation Debt" in Machine Learning Research: A Retrospective Datasheet for BookCorpus* 2021. arXiv: 2105.05241 [cs.CL]. https://arxiv.org/abs/2105.05241.

82. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research* **15,** 1929–1958 (2014).

83. Ba, J. L., Kiros, J. R. & Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450* (2016).

84. Loshchilov, I. & Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

85. Horner, W. *A New Method of Solving Numerical Equations of all Orders, by Continuous Approximation.* in *Abstracts of the Papers Printed in the Philosophical Transactions of the Royal Society of London* **2** (1815), 117–117.

86. Nickolls, J., Buck, I., Garland, M. & Skadron, K. Scalable parallel programming with cuda: Is cuda the parallel programming model that application developers have been waiting for? *Queue* **6,** 40–53 (2008).

87. Kumar, S. K. On weight initialization in deep neural networks. *arXiv preprint arXiv:1704.08863* (2017).

88. Bodner, A. *Convolutional-KANs* 2025. https://github.com/AntonioTepsich/Convolutional-KANs.

A Appendix

A.1 Code

```
class KANLinear(torch.nn.Module):
    def __init__(
        self,
        in_features,
        out_features,
        grid_size=2,
        spline_order=2,
        scale_noise=0.1,
        scale_base=1.0,
        scale_spline=1.0,
        enable_standalone_scale_spline=True,
        base_activation=torch.nn.GELU,
        grid_eps=0.02,
        grid_range=[-1, 1],
    ):
        super(KANLinear, self).__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.grid_size = grid_size
        self.spline_order = spline_order

        h = (grid_range[1] - grid_range[0]) / grid_size
        grid = (
            (
                torch.arange(-spline_order, grid_size + spline_order + 1) * h
                + grid_range[0]
            )
            .expand(in_features, -1)
            .contiguous()
        )
        self.register_buffer("grid", grid)

        self.base_weight = torch.nn.Parameter(torch.Tensor(out_features, in_features))
```

```
        self.spline_weight = torch.nn.Parameter(
            torch.Tensor(out_features, in_features, grid_size + spline_order)
        )
        if enable_standalone_scale_spline:
            self.spline_scaler = torch.nn.Parameter(
                torch.Tensor(out_features, in_features)
            )

        self.scale_noise = scale_noise
        self.scale_base = scale_base
        self.scale_spline = scale_spline
        self.enable_standalone_scale_spline = enable_standalone_scale_spline
        self.base_activation = base_activation()
        self.grid_eps = grid_eps

        self.reset_parameters()

    def reset_parameters(self):
        torch.nn.init.kaiming_uniform_(self.base_weight, a=math.sqrt(5) * self.scale_base)
        with torch.no_grad():
            noise = (
                (
                    torch.rand(self.grid_size + 1, self.in_features, self.out_features)
                    - 1 / 2
                )
                * self.scale_noise
                / self.grid_size
            )
            self.spline_weight.data.copy_(
                (self.scale_spline if not self.enable_standalone_scale_spline else 1.0)
                * self.curve2coeff(
                    self.grid.T[self.spline_order : -self.spline_order],
                    noise,
                )
            )
            if self.enable_standalone_scale_spline:
                # torch.nn.init.constant_(self.spline_scaler, self.scale_spline)
                torch.nn.init.kaiming_uniform_(self.spline_scaler, a=math.sqrt(5) * self.scale_spline)

```

```
    def b_splines(self, x: torch.Tensor):
        """
        Compute the B-spline bases for the given input tensor.

        Args:
            x (torch.Tensor): Input tensor of shape (batch_size, in_features).

        Returns:
            torch.Tensor: B-spline bases tensor of shape (batch_size, in_features, grid_size + spline_order).
        """
        assert x.dim() == 2 and x.size(1) == self.in_features

        grid: torch.Tensor = (
            self.grid
        )  # (in_features, grid_size + 2 * spline_order + 1)
        x = x.unsqueeze(-1)
        bases = ((x >= grid[:, :-1]) & (x < grid[:, 1:])).to(x.dtype)
        for k in range(1, self.spline_order + 1):
            bases = (
                (x - grid[:, : -(k + 1)])
                / (grid[:, k:-1] - grid[:, : -(k + 1)])
                * bases[:, :, :-1]
            ) + (
                (grid[:, k + 1 :] - x)
                / (grid[:, k + 1 :] - grid[:, 1:(-k)])
                * bases[:, :, 1:]
            )

        assert bases.size() == (
            x.size(0),
            self.in_features,
            self.grid_size + self.spline_order,
        )
        return bases.contiguous()

    def curve2coeff(self, x: torch.Tensor, y: torch.Tensor):
        """
        Compute the coefficients of the curve that interpolates the given points.

        Args:
            x (torch.Tensor): Input tensor of shape (batch_size, in_features).
            y (torch.Tensor): Output tensor of shape (batch_size,
```

```
    in_features, out_features).

        Returns:
            torch.Tensor: Coefficients tensor of shape (
    out_features, in_features, grid_size + spline_order).
        """
        assert x.dim() == 2 and x.size(1) == self.in_features
        assert y.size() == (x.size(0), self.in_features, self.
    out_features)

        A = self.b_splines(x).transpose(
            0, 1
        )  # (in_features, batch_size, grid_size + spline_order)
        B = y.transpose(0, 1)  # (in_features, batch_size,
    out_features)
        solution = torch.linalg.lstsq(
            A, B
        ).solution  # (in_features, grid_size + spline_order,
    out_features)
        result = solution.permute(
            2, 0, 1
        )  # (out_features, in_features, grid_size + spline_order)

        assert result.size() == (
            self.out_features,
            self.in_features,
            self.grid_size + self.spline_order,
        )
        return result.contiguous()

    @property
    def scaled_spline_weight(self):
        return self.spline_weight * (
            self.spline_scaler.unsqueeze(-1)
            if self.enable_standalone_scale_spline
            else 1.0
        )

    def forward(self, x: torch.Tensor):
        assert x.size(-1) == self.in_features
        original_shape = x.shape
        x = x.view(-1, self.in_features)

        base_output = F.linear(self.base_activation(x), self.
    base_weight)
        spline_output = F.linear(
            self.b_splines(x).view(x.size(0), -1),
```

```
            self.scaled_spline_weight.view(self.out_features, -1),
        )
        output = base_output + spline_output

        output = output.view(*original_shape[:-1], self.
out_features)
        return output

    @torch.no_grad()
    def update_grid(self, x: torch.Tensor, margin=0.01):
        assert x.dim() == 2 and x.size(1) == self.in_features
        batch = x.size(0)

        splines = self.b_splines(x)  # (batch, in, coeff)
        splines = splines.permute(1, 0, 2)  # (in, batch, coeff)
        orig_coeff = self.scaled_spline_weight  # (out, in, coeff)
        orig_coeff = orig_coeff.permute(1, 2, 0)  # (in, coeff, out
)
        unreduced_spline_output = torch.bmm(splines, orig_coeff)  #
 (in, batch, out)
        unreduced_spline_output = unreduced_spline_output.permute(
            1, 0, 2
        )  # (batch, in, out)

        # sort each channel individually to collect data
distribution
        x_sorted = torch.sort(x, dim=0)[0]
        grid_adaptive = x_sorted[
            torch.linspace(
                0, batch - 1, self.grid_size + 1, dtype=torch.int64
, device=x.device
            )
        ]

        uniform_step = (x_sorted[-1] - x_sorted[0] + 2 * margin) /
self.grid_size
        grid_uniform = (
            torch.arange(
                self.grid_size + 1, dtype=torch.float32, device=x.
device
            ).unsqueeze(1)
            * uniform_step
            + x_sorted[0]
            - margin
        )

        grid = self.grid_eps * grid_uniform + (1 - self.grid_eps) *
```

```
    grid_adaptive
        grid = torch.concatenate(
            [
                grid[:1]
                - uniform_step
                * torch.arange(self.spline_order, 0, -1, device=x.
    device).unsqueeze(1),
                grid,
                grid[-1:]
                + uniform_step
                * torch.arange(1, self.spline_order + 1, device=x.
    device).unsqueeze(1),
            ],
            dim=0,
        )

        self.grid.copy_(grid.T)
        self.spline_weight.data.copy_(self.curve2coeff(x,
    unreduced_spline_output))

    def regularization_loss(self, regularize_activation=1.0,
    regularize_entropy=1.0):
        l1= self.spline_weight.abs().mean(-1)
        regularization_loss_activation = l1.sum()
        p = l1 / regularization_loss_activation
        regularization_loss_entropy = -torch.sum(p * p.log())
        return (
            regularize_activation * regularization_loss_activation
            + regularize_entropy * regularization_loss_entropy
        )
```

Listing A.1: The KAN layer class [88]

```
class RationalFunction(nn.Module):
    """
    Implements the rational base function used in GR-KAN.
    The function has the form: F(x) = P(x) / (1 + |Q(x)|)
    where P and Q are polynomials.
    """
    def __init__(self, degree_num=5, degree_den=4, group_size=1):
        super().__init__()
        self.degree_num = degree_num
        self.degree_den = degree_den
        self.group_size = group_size

        # Coefficients for numerator (P(x)) and denominator (Q(x))
        self.a = nn.Parameter(torch.zeros(group_size, degree_num +
    1))  # +1 for constant term
```

```
        self.b = nn.Parameter(torch.zeros(group_size, degree_den))
        # no constant term in denominator

        self.initialize_coefficients()

    def initialize_coefficients(self):
        """Initialize coefficients to approximate identity function
    """
        nn.init.constant_(self.a[:, 0], 0.0)  # constant term
        nn.init.constant_(self.a[:, 1], 1.0)  # linear term
        nn.init.constant_(self.a[:, 2:], 0.0)  # higher order terms

        # Initialize denominator to small values
        nn.init.normal_(self.b, mean=0.0, std=0.1)

    def forward(self, x):
        """
        Compute rational function using Horner's method for
    efficiency
        x: input tensor of shape (..., group_size)
        returns: output tensor of same shape as input
        """
        # Reshape for group processing
        orig_shape = x.shape
        x = x.view(-1, self.group_size)

        numerator = self.a[:, 0]
        for i in range(1, self.degree_num + 1):
            numerator = numerator + self.a[:, i] * (x ** i)

        denominator = torch.zeros_like(x)
        for i in range(1, self.degree_den + 1):
            denominator = denominator + self.b[:, i-1] * (x ** i)

        output = numerator / (1 + torch.abs(denominator))

        return output.view(*orig_shape)
```

Listing A.2: Rational Function implementation

```
class KAT_Group(nn.Module):
    """
    Group-Rational KAN (GR-KAN) layer that replaces MLP in
    Transformers.
    Implements the equation: GR-KAN(x) = W * F(x)
    where F is the group-wise rational function.
    """
    def __init__(self, dim_in, dim_out, num_groups=8, degree_num=5,
```

```
    degree_den=4):
        super().__init__()
        self.dim_in = dim_in
        self.dim_out = dim_out
        self.num_groups = num_groups
        self.group_size = dim_in // num_groups

        assert dim_in % num_groups == 0

        self.rational_fn = RationalFunction(degree_num, degree_den,
    num_groups)

        self.weight = nn.Parameter(torch.empty(dim_out, dim_in))
        self.bias = nn.Parameter(torch.empty(dim_out))

        self.reset_parameters()

    def reset_parameters(self):
        """Initialize weights using variance-preserving
    initialization"""
        gain = 1.0  # This should be adjusted based on actual
    rational function behavior

        nn.init.xavier_uniform_(self.weight, gain=math.sqrt(gain /
    self.dim_in))

        if self.bias is not None:
            nn.init.zeros_(self.bias)

    def forward(self, x):
        orig_shape = x.shape
        if len(orig_shape) == 3:
            batch_size, seq_len, dim_in = orig_shape
            x = x.reshape(batch_size * seq_len, dim_in)
        elif len(orig_shape) == 2:
            batch_size, dim_in = orig_shape
            seq_len = 1
        else:
            raise ValueError(f"Unexpected input shape: {orig_shape}
    ")
        x = x.view(-1, self.num_groups, self.group_size)

        fx = self.rational_fn(x)

        fx = fx.view(*orig_shape)

        output = F.linear(fx, self.weight, self.bias)
```

```
49
50          return output
51
52      def dimensions(self):
53          return f"dim_in={self.dim_in}, dim_out={self.dim_out},
    num_groups={self.num_groups}"
```

Listing A.3: The GR-KAN layer implemetation